小组实验报告

数据备份软件（DataBackup）

组长: 王科瑾
学号: 待填写

目录

一、实验室名称.....4
二、实验项目名称.....4
三、实验目的.....4
四、实验内容.....4
五、实验器材（设备、元器件）.....5
六、实验步骤及操作.....6
需求分析说明书.....7
1. 任务概述.....7
1.1 引言.....7
1.2 综合描述.....7
1.3 运行环境.....10
2. 功能需求.....10
2.1 功能划分.....10
2.2 系统用例.....10
3. 外部接口需求.....24
3.1 用户界面.....24
3.2 硬件接口.....25
3.3 软件接口.....25
4. 其它非功能性需求.....26
4.1 性能需求.....26

4.2 安全性需求.....26
4.3 软件质量属性.....26
5. 项目规划.....26
5.1 人员分工.....26
5.2 项目总体规划.....27
系统设计文档.....28
1. 开发环境和工具.....28
1.1 开发环境.....28
1.2 依赖库.....28
2. 总体设计.....29
2.1 系统结构设计.....29
3. 静态建模.....33
3.1 系统对象模型.....33
3.2 类（对象）描述.....34
3.3 其它静态建模方式（可选）.....47
4. 动态建模.....48
4.1 系统场景.....48
4.2 系统状态.....51
5. 数据库设计（如有）.....53
软件测试报告.....55
1. 引言.....55
1.1 测试目的与范围.....55
1.2 测试环境.....55
1.3 测试类型与规模.....56
2. 功能测试.....56

2.1 密码学原语模块（internal/crypto，FR-13 底层）.....56
2.2 压缩模块（internal/huffman，FR-12）.....58
2.3 打包模块（internal/pack，FR-11、FR-17）.....58
2.4 容器模块（internal/container，FR-13）.....59
2.5 过滤模块（internal/filter，FR-05 ~ FR-10）.....60
2.6 遍历与元数据模块（internal/walk、internal/fsmeta，FR-01、FR-03、FR-04）.....60
2.7 引擎集成（internal/engine，FR-01 ~ FR-04）.....62
2.8 仓库管理与配置（internal/repo、internal/config，FR-14、FR-15）.....62
2.9 CLI 黑盒（testscript，FR-01 ~ FR-15）.....63
2.10 客户端（FR-16、FR-18）.....65
3. 代码测试.....68
3.1 静态检查（M8-4，2026-10-17 实测）.....68
3.2 覆盖率（M8-2，2026-10-17 实测，go test -coverprofile）.....68
3.3 竞态与模糊测试（M8-2）.....69
4. 性能测试（M8-3，2026-10-17 实测）.....69
4.1 微基准（go test -bench，benchtime=2s，开发容器 Ubuntu 22.04 / go1.27.1）.....69
4.2 场景吞吐与压缩率（WSL2 Ubuntu 24.04，真实二进制，NVMe）.....70
4.3 性能剖析（pprof）.....71
5. 健壮性测试.....71
6. 测试结果分析.....74
6.1 汇总.....74
6.2 FR ↔ TC 追踪表.....74
6.3 结论与已知限制.....76

七、AI 开发工具使用总结.....77
7.1 使用的 AI 工具.....77
7.2 使用方法与效果评价.....77
八、实验结论.....78
九、总结及心得体会.....79
十、对本实验过程及方法、手段的改进建议.....79

一、实验室名称

信息与软件工程学院实验室（以课程课件标注为准）

二、实验项目名称

文件备份软件（DataBackup）

三、实验目的

1. 掌握软件开发全生命周期过程与方法：按面向对象软件工程方法完成需求分析 → 系统设计 → 编码实现 → 软件测试 → 打包发布的完整流程，产出规范的过程文档与可运行的软件系统。
2. 掌握现代工程化开发环境：Linux 下 Go 编译型后端与 Electron 桌面客户端的跨端协作开发，容器化统一工具链（Docker 开发镜像、make、Git 分支与 CI 门禁、GitHub Actions）。
3. 掌握备份领域关键技术并自研实现：目录树遍历与过滤、自研归档格式（MyPack）、Huffman 压缩、AES-256 加密（全部手写，不使用现成库）。
4. 实践 AI 辅助开发：以多编码 agent 并行分工的方式完成开发任务，并对 AI 产出建立审查与质量门禁。

四、实验内容

基础要求（40 分）：数据备份——将目录树中的文件数据保存到指定位置；数据还原——恢复到指定位置。

扩展要求（以课件 5/6 新版为准，各扩展项计分见 `docs/project/course-requirements.md` §2.2）：

扩展项	分值	本组选择
文件类型支持（管道/软链接/硬链接/设备等）	10	☒ FR-03：普通文件、目录、软链接、硬链接、FIFO、字符/块设备、套接字共 7 种
元数据支持（属主/时间/权限等）	10	☒ FR-04：权限位（含 setuid/setgid/sticky）、uid/gid、纳秒级 atime/mtime
自定义备份（路径/类型/名字/时间/尺寸 各 5 分）	5×5	☒ FR-05~09 全做；另实现旧版要求的用户/组过滤（FR-10）
打包解包	10	☒ FR-11：自研 MyPack 归档格式（512 字节头记录，仿 tar）
压缩解压	10	☒ FR-12：自研 Huffman 编码（1MiB 分块流式）
加密解密	10	☒ FR-13：手写 PBKDF2-HMAC-SHA256 + AES-256-CBC + HMAC 校验
图形界面	10	☒ FR-16：Electron + React 桌面客户端（备份/管理/设置三页）

另实现支撑功能：备份仓库管理（FR-14）、配置（FR-15）、安全健壮性（FR-17：完整性校验、路径穿越防护、错误口令识别）、进度与日志（FR-18：JSON Lines 协议实时展示）。功能需求清单详见 `01-需求分析说明书.md` §2，FR-01~18 全部实现。

五、实验器材（设备、元器件）

PC 多台，可连接互联网。开发环境：

- 开发主机：Windows 11 + Git Bash + Docker Desktop（统一开发镜像 `databackup-dev`：Ubuntu 22.04 + go1.27.1 + Node.js 22.23.3 +

staticcheck 2026.2.1，版本锁定并 SHA256 校验，`scripts/install-deps.sh` 一键复现）。

- 运行/演示环境：WSL2 Ubuntu 24.04 + WSLg（图形客户端演示）；CI：GitHub Actions（Ubuntu 22.04）。
- 版本管理与质量门禁：Git 分支 + PR、make check / check-all、Playwright 截图回归基线。

六、实验步骤及操作

本章由 01-需求分析说明书、02-系统设计文档、03-软件测试报告 三部分组成，合成《小组实验报告》时依次拼接在此标题之后。

需求分析说明书

1. 任务概述

1.1 引言

编写目的：本文档是 DataBackup 数据备份软件的需求规格说明，作为后续系统设计（《系统设计文档》）、编码实现与软件测试（《软件测试报告》）的基线。读者为本项目开发者（组长王科瑾）、课程评审教师与答辩委员。

术语：

术语	含义
备份仓库（repo）	存放备份文件（`.dbk`）的目录；系统不使用数据库，仓库状态由扫描 `.dbk` 文件头得出
MyPack	本项目自研的归档格式（仿 tar：512 字节头记录 + 数据块）
JSONL / JSON Lines	后端 `--json` 模式下在 stdout 上逐行输出 JSON 事件的协议
哨兵错误	Go 中用于 `errors.Is` 判别的包级错误值，CLI 据此映射为错误码与退出码
FR-xx	功能需求编号，与《项目简介》（`../project/overview.md`）§3 一致
UC-xx	用例编号，见 §2.2
契约	`../architecture/cli-contract.md`，后端 CLI/JSONL 协议的唯一来源（已冻结）

参考资料：课程任务书（`../project/course-requirements.md`）、项目简介（`../project/overview.md`）、架构设计（`../architecture/overview.md`）、CLI 协议（`../architecture/cli-contract.md`）。

1.2 综合描述

1.2.1 产品的状况

DataBackup 是新型、自主开发的单机 Linux 数据备份工具，由两部分组成：

- **后端**：Go 编写的命令行程序 `databackup`，负责备份、还原、过滤、打包、压缩、加密、备份管理与配置，仅以标准库实现（自研 MyPack 归档、Huffman 压缩、SHA-256/HMAC/PBKDF2/AES-256-CBC 加密）。
- **客户端**：Electron + React 桌面程序，通过 spawn 子进程调用后端 CLI 并以 JSON Lines 协议接收进度与结果，提供"备份 / 管理备份数据 / 设置"三个页面。

产品不依赖网络、数据库与第三方运行时，可完全离线运行。

1.2.2 产品的功能

编号	功能	说明
FR-01	数据备份	将源目录树备份为一个 `.dbk` 备份文件，存入备份仓库
FR-02	数据还原	将备份文件还原到指定（不存在的或空的）目录
FR-03	文件类型支持	普通文件、目录、软链接、硬链接、FIFO、字符设备、块设备、套接字
FR-04	元数据支持	权限位、属主/属组（uid/gid）、访问/修改时间，还原时恢复
FR-05	过滤：路径	选择源目录，排除指定子目录
FR-06	过滤：类型	按文件类型（`f d l c b s p`）筛选
FR-07	过滤：名字	按正则（RE2）排除文件名
FR-08	过滤：时间	按修改时间 / 创建（状态变更）时间闭区间筛选
FR-09	过滤：尺寸	按文件大小闭区间筛选（仅普通文件）
FR-10	过滤：用户/组	排除指定属主或属组的文件

编号	功能	说明
FR-11	打包/解包	自研 MyPack 归档格式
FR-12	压缩/解压	自研规范 Huffman 编码（分块流式）
FR-13	加密/解密	口令 → PBKDF2-HMAC-SHA256 → AES-256-CBC + HMAC 校验（全部手写）
FR-14	备份管理	列出、查看、删除仓库中的备份
FR-15	设置	配置备份仓库路径等
FR-16	图形客户端	Electron 客户端三个页面
FR-17	安全与健壮性	完整性校验、错误口令识别、目标目录须为空、拒绝路径穿越、错误输入友好提示
FR-18	进度与日志	后端 JSONL 输出进度/日志/结果，客户端实时展示

过滤、还原与错误处理的精确语义以已冻结的 `../architecture/cli-contract.md` §2、§4 为准，本文不另行定义。

1.2.3 用户类和特性

用户类	特征	主要使用方式
普通桌面用户	需要备份个人目录（文档、照片等），不一定熟悉命令行	Electron 客户端三页操作
进阶/运维用户	熟悉 shell，可能将备份纳入脚本	直接调用 `databackup` CLI（`--json` 输出供程序消费）

两类用户共用同一个后端，功能与语义完全一致。

1.3 运行环境

1.3.1 基本配置

（1）硬件平台：x86-64 个人计算机。

（2）操作系统：Linux（目标发行版 Ubuntu 22.04 / 24.04）；演示环境可为 WSL2 Ubuntu（WSLg 显示 Electron 窗口）。

1.3.2 其他配置

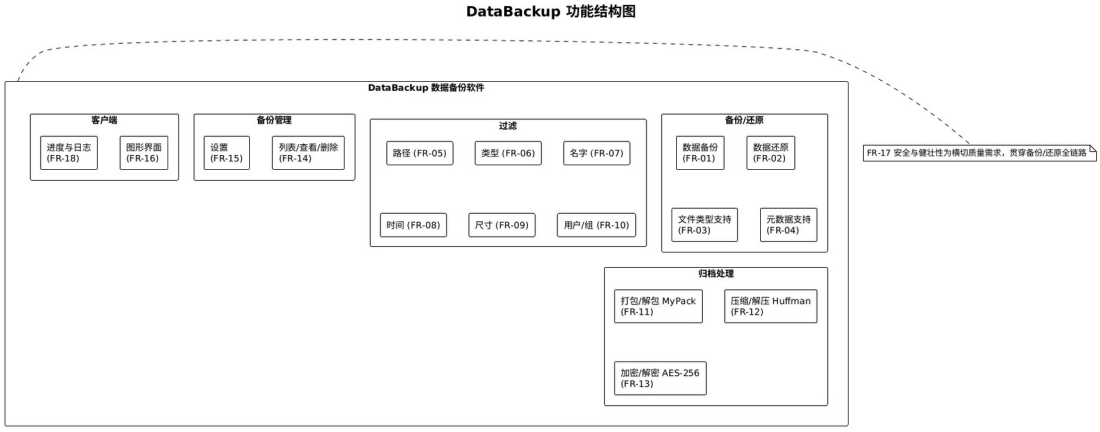
（1）内存：⩾ 512 MB（后端流式处理，内存占用与文件总量无关）。

（2）硬盘：除系统占用外，需要与备份数据量相当的空闲空间存放 `.dbk` 文件。

（3）网络：不需要。

2. 功能需求

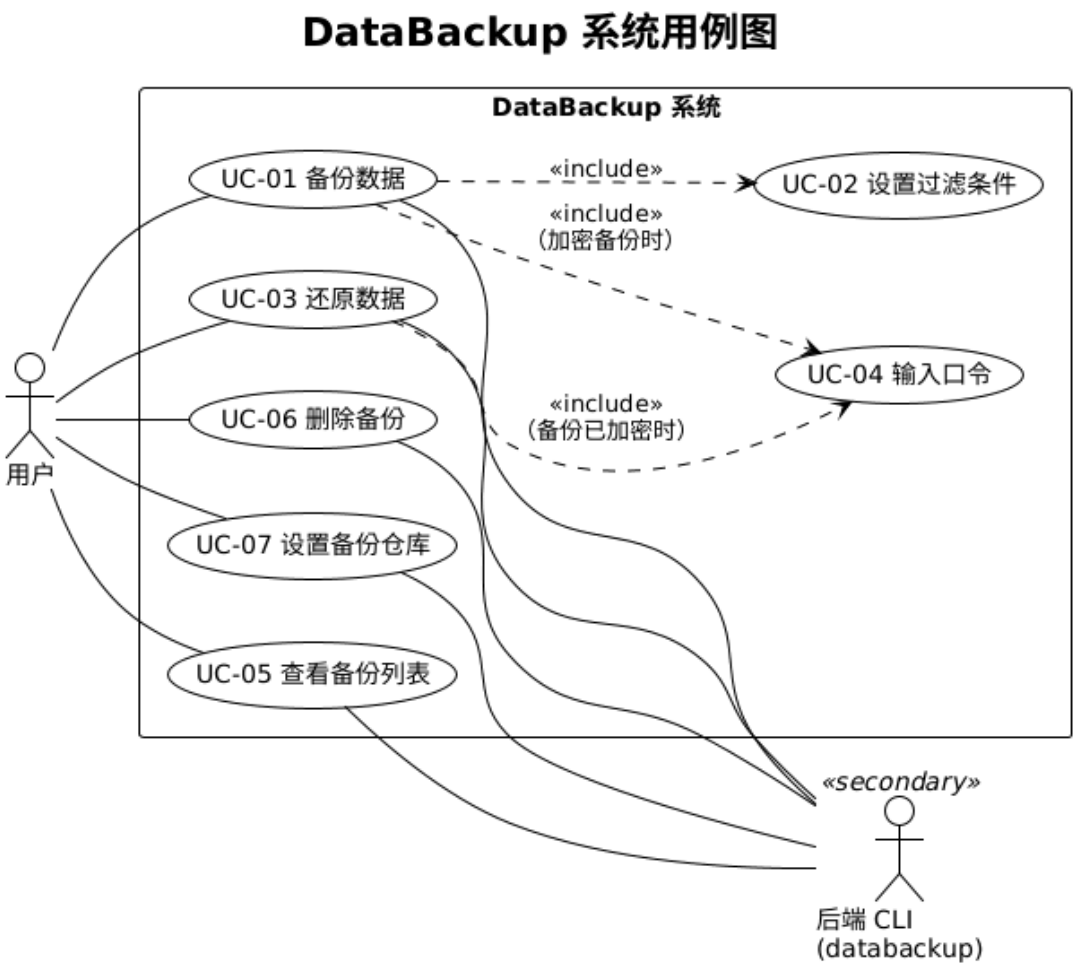
2.1 功能划分



（源文件 `diagrams/function-structure.puml`。）功能按"备份/还原、过滤、归档处理、备份管理、客户端"五组划分；FR-17（安全与健壮性）为横切质量需求，体现在各组的校验与错误处理中。

2.2 系统用例

系统用例总图如下（各用例小节的用例图均为该图对应部分）：



（源文件 `diagrams/use-cases.puml`。）

2.2.1 用例编号 UC-01

2.2.1.1 用例图

见系统用例总图：UC-01 备份数据（include UC-02；加密时 include UC-04）。

2.2.1.2 用例描述

字段	内容
用例标识	UC-01
用例名称	备份数据
创建人	王科瑾
创建日期	2026-10-16
语境目标	将源目录树按选定条件备份为仓库中的一个 `.dbk` 文件（FR-01、03、04、11、12、13、17、1

字段	内容
	8）
前置条件	源目录存在且为目录；备份名合法（1~200 字节、不含 / 与 NUL、不以 . 开头）且不重名
成功的结束状态	仓库中新增 <name>.dbk；输出结果含文件/目录数、原始与输出字节数、警告数
失败的结束状态	不产出 .dbk（.partial 已删除）；输出对应错误码的 error 事件或 stderr 诊断
参与者	用户（经客户端或 CLI）、后端 databackup
触发器	用户在"备份"页点击"开始备份"，或执行 databackup backup
基础用例	无
包含用例	UC-02（设置过滤条件）；选择 AES-256 加密时 UC-04（输入口令）
扩展用例	预览（--dry-run）：只扫描统计、不写文件、不读口令
事件流	基本流程：1) 用户选择源目录与可选参数（名字、压缩、加密、过滤条件）；2) 系统校验参数（非法 → E_USAGE）；3) 用户确认开始；4) 系统先扫描（scan 阶段进度事件）再打包写出 .partial（pack 阶段进度事件）；5) 成功后原子改名为 .dbk 并输出结果事件。扩展流程：2a 参数非法 → 输出 E_USAGE、退出码 2；2b 源目录不存在 → E_NOT_FOUND、退出码 1；4a 同名备份已存在 → E_EXISTS；4b 用户取消 → 删除 .partial、E_CANCELLED、退出码 4；4c IO/校验错误 → 删除 .partial、输出 E_IO/E_CORRUPT 等；4d 其他

字段	内容
	未分类的内部错误 → 删除 `.partial`、E_INTERNAL、退出码 1；4e 某些项无法读取（权限不足等）→ 输出 warn 并跳过、计入 `warnings`，备份继续
非功能需求	流式处理，不整体读入内存；进度事件每阶段 ≤10 次/秒且阶段结束必发一次（FR-18）
补充规格说明书	参数、过滤语义、事件字段与错误码详见 `../architecture/cli-contract.md` §2.1、§3、§4

2.2.2 用例编号 UC-02

2.2.2.1 用例图

见系统用例总图：UC-02 设置过滤条件（被 UC-01 include）。

2.2.2.2 用例描述

字段	内容
用例标识	UC-02
用例名称	设置过滤条件
创建人	王科瑾
创建日期	2026-10-16
语境目标	限定备份范围：类型、排除子目录、排除名字正则、时间区间、尺寸区间、排除属主/属组（FR-05~10）
前置条件	处于备份参数编辑中
成功的结束状态	过滤条件被接受并用于本次备份/预览
失败的结束状态	非法条件被拒，提示 E_USAGE
参与者	用户
触发器	用户在备份表单填写过滤项，或在命令行给出 `--types` 等参数
基础用例	UC-01
包含用例	无

字段	内容
扩展用例	无
事件流	基本流程：1) 用户勾选类型集合 `f d l c b s p`（空集合=全部）；2) 添加排除子目录（可多条，相对源目录或源内绝对路径）；3) 添加排除文件名正则（RE2，只匹配 `path.Base`）；4) 设置修改/创建时间闭区间（缺省不限，只写日期时 from 取 00:00:00、to 取 23:59:59.999999999 本地时区）；5) 设置尺寸闭区间（仅作用于普通文件）；6) 添加排除属主/属组（名称或数字 ID）。扩展流程：任一正则非法、区间反向、用户名/组名不存在 → E_USAGE。目录只受类型与排除目录约束（名字/时间/尺寸/用户组不作用于目录）；排除目录命中即整棵跳过。附加规则（契约 §2.1）：a) 源目录本身不进入归档，归档只含其下内容，路径一律相对源目录；b) 仓库目录若位于源目录内，自动排除；c) 硬链接只在被保留的条目之间识别——同一 inode 第一次被备份时存数据，之后的路径记为指向它的硬链接，被过滤掉的路径不参与识别；d) 无法读取的项（权限不足等）输出 warn 并跳过、计入 `warnings`，不使备份失败
非功能需求	校验在参数解析与 `filter.New` 两级完成，尽早报 E_USAGE
补充规格说明书	过滤语义以 `../architecture/cli-contract.md` §2.1"过滤语义"为准

2.2.3 用例编号 UC-03

2.2.3.1 用例图

见系统用例总图：UC-03 还原数据（备份已加密时 include UC-04）。

2.2.3.2 用例描述

字段	内容
用例标识	UC-03
用例名称	还原数据
创建人	王科瑾
创建日期	2026-10-16
语境目标	将 `.dbk` 备份还原为目录树，恢复文件类型与元数据（FR-02、03、04、11、12、13、17、18）
前置条件	备份文件存在；目标目录不存在或为空
成功的结束状态	目标目录中重建出与备份一致的目录树（类型、权限、属主、时间戳）
失败的结束状态	目标目录可能不完整（取消不回滚，发 warn 说明）；输出错误事件
参与者	用户（经客户端或 CLI）、后端 `databackup`
触发器	用户在"管理备份数据"页选择备份点"还原"，或执行 `databackup restore`
基础用例	无
包含用例	备份已加密时 UC-04（输入口令）
扩展用例	无
事件流	基本流程：1) 用户选择备份（名称或 `.dbk` 路径）与目标目录；2) 系统解析备份位置（不存在 → E_NOT_FOUND）；3) 检查目标目录（已存在且非空或不是目录 → E_TARGET_NOT_EMPTY、退出码 1）；4) 写任何文件前完整校验备份（损坏 → E_CORRUPT；加密未给口令 → E_PASSWORD_REQUIRED、退出码 3；口令错误/被篡改 →

字段	内容
	E_AUTH、退出码 3）；5) 逐条重建文件并报告进度（restore 阶段）；6) 输出结果。扩展流程：5a 容器内条目含 `../`绝对路径 → 拒绝、E_CORRUPT（路径穿越防护，FR-17）；5b 非 root 用户 → 跳过属主恢复并输出 warn，不视为失败；无法创建的字符/块设备同样 warn 后跳过（计入 warnings、不计入 files）；5c 用户取消 → 不回滚、warn 提示目标不完整、E_CANCELLED、退出码 4；5d 其他未分类的内部错误 → E_INTERNAL、退出码 1
非功能需求	还原先完整校验再写文件；目录元数据最后设置
补充规格说明书	还原与错误语义以 `../architecture/cli-contract.md` §2.2、§4 为准

2.2.4 用例编号 UC-04

2.2.4.1 用例图

见系统用例总图：UC-04 输入口令（被 UC-01/UC-03 include）。

2.2.4.2 用例描述

字段	内容
用例标识	UC-04
用例名称	输入口令
创建人	王科瑾
创建日期	2026-10-16
语境目标	为加密备份提供/验证口令（FR-13、17）
前置条件	操作对象需要口令：备份选择 `--encrypt aes256`，或还原的备份已加密
成功的结束状态	口令被后端接受（备份）或校验通过（还原）

字段	内容
失败的结束状态	口令为空或超 1024 字节 → E_USAGE；还原时口令错误 → E_AUTH；客户端连续 3 次失败后停止重试、返回就绪态（对话框提示剩余次数）
参与者	用户
触发器	用户勾选加密 / 还原加密备份；命令行为 `--password-stdin` 从 stdin 读入
基础用例	UC-01、UC-03
包含用例	无
扩展用例	无
事件流	基本流程：1) 客户端弹出口令输入框（不回显）或在脚本中经 stdin 提供；2) 口令经 stdin 传给后端，读到首个 `\n` 或 EOF 为止、去结尾 `\r`，长度 1~1024 字节；3) 后端用于 PBKDF2 派生密钥，不记录、不回显、不出现在命令行参数/环境变量/日志中。扩展流程：口令为空或超长 → E_USAGE；`--encrypt aes256` 与 `--password-stdin` 未配套 → E_USAGE；还原口令错误 → 客户端回到口令对话框并提示剩余次数，连续 3 次失败后回到就绪态（`../client/interaction.md` §3）
非功能需求	口令只允许经 stdin 传递（FR-17）；客户端不持久化口令
补充规格说明书	口令输入规则见 `../architecture/cli-contract.md` §1"约定"

2.2.5 用例编号 UC-05

2.2.5.1 用例图

见系统用例总图：UC-05 查看备份列表。

2.2.5.2 用例描述

字段	内容
用例标识	UC-05
用例名称	查看备份列表
创建人	王科瑾
创建日期	2026-10-16
语境目标	列出仓库中的全部备份并查看单个备份详情（FR-14）
前置条件	已确定仓库目录（`--repo` → 配置文件 → `~/DataBackup`）
成功的结束状态	按创建时间降序、名字升序列出 `.dbk` 备份；可查看单个备份的头/尾信息
失败的结束状态	仓库不可读等 → E_IO；查看不存在/损坏的备份 → E_NOT_FOUND/E_CORRUPT
参与者	用户（经客户端或 CLI）、后端 `databackup`
触发器	打开"管理备份数据"页，或执行 `databackup list` / `databackup info`
基础用例	无
包含用例	无
扩展用例	无
事件流	基本流程：1) 用户打开管理页（客户端周期性调用 `list`）；2) 系统扫描仓库 `.dbk` 并读取容器头尾（无数据库）；3) 损坏文件以 `corrupt=true` 列出；4) 用户选中某备份查看详情（`info`）：名字、大小、创建时间、来源目录、文件/目录/字节数、是否压缩/加密、格式版本
非功能需求	列出操作只读头尾元数据，不加载负载
补充规格说明书	`list`/`info` 字段与排序见 `../architecture/cli-`

字段	内容
	`contract.md` §3

2.2.6 用例编号 UC-06

2.2.6.1 用例图

见系统用例总图：UC-06 删除备份。

2.2.6.2 用例描述

字段	内容
用例标识	UC-06
用例名称	删除备份
创建人	王科瑾
创建日期	2026-10-16
语境目标	删除仓库中的一个 `.dbk` 备份（FR-14）
前置条件	备份存在
成功的结束状态	该 `.dbk` 文件被删除
失败的结束状态	备份不存在 → E_NOT_FOUND；显式路径不是 `.dbk` → E_USAGE
参与者	用户（经客户端或 CLI）、后端 `databackup`
触发器	用户在管理页对某备份点"删除"（客户端先弹确认），或执行 `databackup delete`
基础用例	无
包含用例	无
扩展用例	无
事件流	基本流程：1) 用户选择备份（名称或 `.dbk` 路径）；2) 客户端弹出确认对话框；3) 系统删除文件并返回结果（只删除 `.dbk`，拒绝其他路径）；4) 列表刷新。扩展流程：备份不存在 → E_NOT_FOUND
非功能需求	删除操作不可恢复，客户端必须二次确认
补充规格说明书	名称解析与只删 `.dbk` 的约束见 `../architecture/cli-`

字段	内容
	`contract.md` §2.2

2.2.7 用例编号 UC-07

2.2.7.1 用例图

见系统用例总图：UC-07 设置备份仓库。

2.2.7.2 用例描述

字段	内容
用例标识	UC-07
用例名称	设置备份仓库路径
创建人	王科瑾
创建日期	2026-10-16
语境目标	查看/修改备份仓库默认路径（FR-15）
前置条件	无
成功的结束状态	`config.json` 中原子更新 `repo`，后续备份/管理默认使用新路径
失败的结束状态	值为相对路径或未知 KEY → E_USAGE；写入失败 → E_IO
参与者	用户（经客户端或 CLI）、后端 `databackup`
触发器	用户在"设置"页修改仓库路径，或执行 `databackup config set repo <path>`
基础用例	无
包含用例	无
扩展用例	无
事件流	基本流程：1) 用户打开设置页（`config get` 显示当前值）；2) 输入新的绝对路径；3) 系统校验（绝对路径；已存在必须是目录）；4) 经同目录临时文件原子写入 `$XDG_CONFIG_HOME/databackup/config.json`；5) 返回完整配置。扩展流程：相对路径/未知 KEY →

字段	内容
	E_USAGE
非功能需求	并发写不产生损坏文件（原子改名）
补充规格说明书	配置语义见 `../architecture/cli-contract.md` §2.2 `config` 命令

2.2.8 功能需求与用例追溯矩阵

● 表示该用例直接实现该需求；○ 表示经 include 关系或作为横切约束涉及该需求。

需求	UC-01 备份	UC-02 过滤	UC-03 还原	UC-04 口令	UC-05 列表	UC-06 删除	UC-07 设置	说明
FR-01 数据备份	●							
FR-02 数据还原			●					
FR-03 文件类型支持	●		●					打包与重建全部 7 种类型及硬链接
FR-04 元数据支持	●		●					备份时采集，还原时恢复
FR-05 过	○	●						UC-01

需求	UC-01 备份	UC-02 过滤	UC-03 还原	UC-04 口令	UC-05 列表	UC-06 删除	UC-07 设置	说明
滤: 路径								include UC-02
FR-06 过滤: 类型	○	●						同上
FR-07 过滤: 名字	○	●						同上
FR-08 过滤: 时间	○	●						同上
FR-09 过滤: 尺寸	○	●						同上
FR-10 过滤: 用户/组	○	●						同上
FR-11 打包/解包	●		●					备份打包、还原解包
FR-12 压缩/解压	●		●					备份压缩、还原解压

需求	UC-01 备份	UC-02 过滤	UC-03 还原	UC-04 口令	UC-05 列表	UC-06 删除	UC-07 设置	说明
FR-13 加密/解密	●		●	●				口令经 UC-04 提供
FR-14 备份管理					●	●		列出/查看、删除
FR-15 设置							●	
FR-16 图形客户端	●	●	●	●	●	●	●	每个用例都可经客户端三页触发
FR-17 安全与健壮性	●	○	●	●	○	○	○	`.partial` 原子改名、HMAC 校验、目标须为空、拒绝路径穿越、只删 `.dbk`

需求	UC-01 备份	UC-02 过滤	UC-03 还原	UC-04 口令	UC-05 列表	UC-06 删除	UC-07 设置	说明
								、配置原子写、非法输入报 E_USAGE
FR-18 进度与日志	●		●					scan/pack/restore 阶段进度与 warn 日志

每个 FR 至少被一个用例覆盖，每个用例至少承载一个 FR；测试用例（TC）与 FR 的追溯见《软件测试报告》。

3. 外部接口需求

3.1 用户界面

- **Electron 客户端**（FR-16）：无边框自绘窗口 + 侧栏导航，三个页面——"备份"（源目录、名字、过滤、压缩、加密表单与进度）、"管理备份数据"（备份列表、详情、还原、删除）、"设置"（仓库路径等）。视觉规范见 `../client/ui-design.md`，交互与状态机见 `../client/interaction.md`。
- **命令行界面**：`databackup <子命令> [参数]`，子命令为 `backup/restore/list/info/delete/config`；`--help`、`--version`、`--json` 为全局参数。文本模式下诊断走 stderr；`--json` 模式下 stdout 只输出 JSON Lines 事件（progress/log/result/error）。

3.2 硬件接口

仅读写本地文件系统；块设备/字符设备仅在"作为备份内容被打包与还原"的意义上涉及（FR-03），不直接操作硬件。

3.3 软件接口

接口	说明
Linux 文件系统 / 系统调用	`lstat/mkfifo/mknod/lchown/link/symlink/utimes` 等，经 Go 标准库 `os/syscall` 使用（FR-03、04）
客户端 ↔ 后端	客户端以 `spawn`（无 shell）调用 `databackup --json <子命令>`，stdout 逐行 JSON 事件，口令经 stdin；协议唯一来源为 `../architecture/cli-contract.md`
错误码与退出码	失败时 stdout 最后一行为 `error` 事件（契约 §4）：退出码 1 —— `E_IO`、`E_NOT_FOUND`、`E_TARGET_NOT_EMPTY`、`E_CORRUPT`、`E_EXISTS`、`E_INTERNAL`（未分类的内部错误）；退出码 2 —— `E_USAGE`；退出码 3 —— `E_AUTH`、`E_PASSWORD_REQUIRED`；退出码 4 —— `E_CANCELLED`。客户端收到非零退出码却没有 `error` 事件时按退出码兜底映射（1 → `E_INTERNAL`，2 → `E_USAGE`，3 → `E_AUTH`，4 或被信号终止 → `E_CANCELLED`）
配置文件	`$XDG_CONFIG_HOME/databackup/config.json`（未设置时 `~/.config/...`），JSON，目前仅 `repo` 一项
备份仓库	目录下若干 `.dbk` 文件；无索引/数据库，列表由扫描容器头尾得出

4. 其它非功能性需求

4.1 性能需求

- 后端流式处理：任意文件以 `io.Reader/Writer` 串联，内存占用不随数据总量增长。
- 进度事件节流：每阶段 ≤10 次/秒，阶段结束必发一次（FR-18）。
- 大目录树（≥10 万项）备份/还原不出现内存性失败；具体吞吐指标在《软件测试报告》中以 Benchmark 证据给出。

4.2 安全性需求

- 口令只经 stdin 传递，不出现在命令行参数、环境变量、日志与持久化存储（FR-17）。
- 加密备份使用 PBKDF2-HMAC-SHA256（10 万轮）派生密钥、AES-256-CBC 加密、HMAC-SHA256 完整性校验（encrypt-then-MAC）；口令错误或数据被篡改报 E_AUTH（FR-13、17）。
- 还原拒绝归档内 `..` 与绝对路径条目（防路径穿越）；备份写 `.partial` 后原子改名，失败/取消不留半成品。
- Electron 保持 `contextIsolation: true`、`sandbox: true`、`nodeIntegration: false`。

4.3 软件质量属性

- **正确性**：备份→还原往返后目录树"五要素"（类型、内容、权限、属主、时间戳）一致，由测试夹具 `AssertTreeEqual` 验证；每类文件类型均有往返用例。
- **健壮性**：损坏/截断的 `.dbk`、错误口令、非空目标目录、非法参数均给出对应错误码而不崩溃；解码器对任意输入不 panic、不死循环（fuzz 覆盖）。
- **易用性**：用法错误统一 E_USAGE 并附用法文本；`--json` 输出可被程序稳定解析；客户端操作均有进度反馈与确认提示。
- **可维护性**：后端零第三方依赖、按包分层（walk→filter→pack→huffman→crypto→container→repo）；CLI 协议冻结并有双端契约测试。

5. 项目规划

5.1 人员分工

本项目为单人组：**组长王科瑾**承担项目经理、需求分析、系统设计、编码、测试全部角色，无组员。项目使用 AI 辅助开发：编码 agent 承担大部分编码、测试与文档起草，组长负责需求与方向、设计决策、验收与现场答辩。

5.2 项目总体规划

项目分波次并行推进（详见 `../roadmap.md` §1）：M0 基建 → M1 接口冻结与文档初稿 → M2~M6 后端各原语与集成 → C0~C3 客户端 → M7 打包 → M8 测试报告 → M9 文档定稿/答辩材料。进度计划见 `../project/schedule.md` 与甘特图 `../project/diagrams/schedule-gantt.png`（源文件 `schedule-gantt.puml`）。

系统设计文档

1. 开发环境和工具

1.1 开发环境

（1）硬件平台：x86-64 个人计算机。

（2）操作系统：开发主机 Windows + Docker Desktop；**构建/测试/演示均在容器或 WSL2 中的 Ubuntu 22.04/24.04**（目标平台 Linux，与课程要求一致）。

（3）编译环境：Go 1.27.1（`go build`/`go test`/`gofmt`/`staticcheck` 2026.2.1）、Node.js 22.23.3 + npm（electron-vite 构建）、GNU Make；统一封装在 `docker/dev.Dockerfile` 开发镜像内（版本由 `scripts/install-deps.sh` 锁定并 SHA256 校验），`bash scripts/docker.sh check` 一键门禁，CI 跑在 GitHub ubuntu-22.04 runner 上，使用同一依赖安装脚本与门禁命令。

（4）IDE：Windsurf（配合 AI 编码 agent）；本项目使用 AI 辅助开发。

1.2 依赖库

后端：仅 Go 标准库，零第三方生产依赖。归档（MyPack）、压缩（Huffman）、密码学（SHA-256、HMAC、PBKDF2、AES-256-CBC）全部自研——课程规定第三方实现的扩展功能减半记分；`scripts/check-imports.sh` 在 lint 阶段强制拦截 `archive/*`、`compress/*`、`crypto/aes|cipher|des|sha*|md5|hmac|pbkdf2`、`golang.org/x/*` 等导入。唯一允许的测试依赖是 `github.com/rogpeppe/go-internal/testscript`（CLI 黑盒测试）。

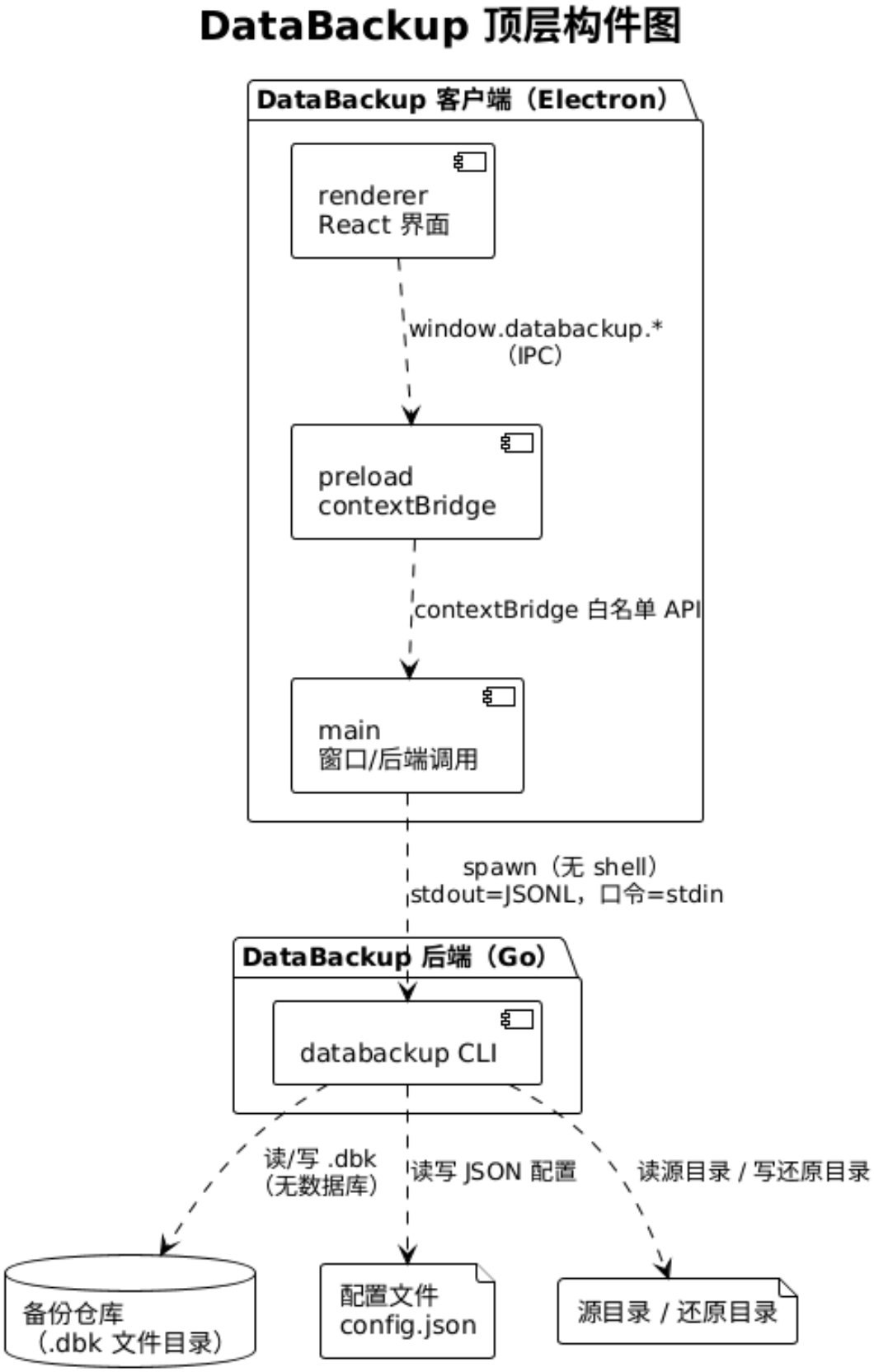
客户端：Electron 44、React 19、TypeScript 5.9、electron-vite 5、electron-builder 26（打包 AppImage/deb）；界面库 Tailwind v4 + shadcn/ui 及配套（zustand、zod、sonner 等，版本在 `client/package.json` 精确锁定）；测试 Vitest 5 + Playwright 1.64。界面库不参与备份逻辑——所有备份/还原/过滤/压缩/加密均由 Go 后端完成，客户端只做界面与进程调用。

2. 总体设计

2.1 系统结构设计

2.1.1 顶层系统结构

2.1.1.1 构件图



（源文件 `diagrams/component-top.puml`。）

2.1.1.2 构件图描述

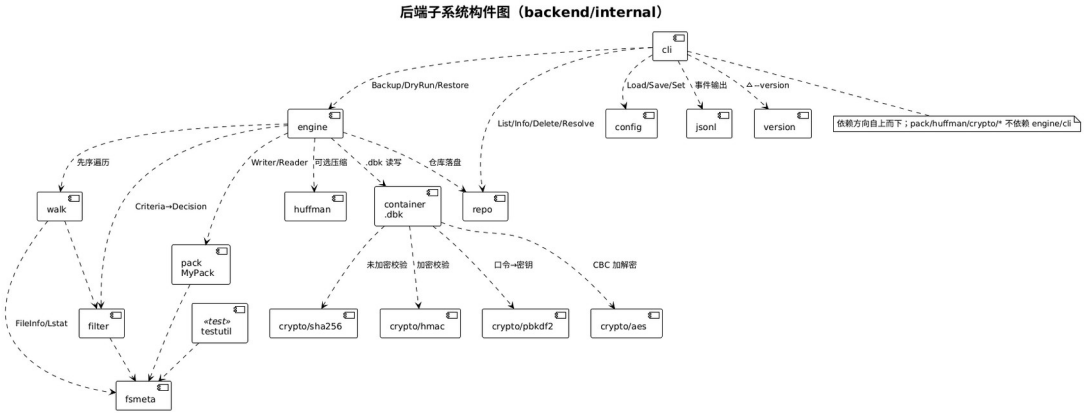
系统由两个可独立部署的构件组成:

- **客户端（Electron）**：renderer（React 界面）只经 `preload` 暴露的 `window.databackup` 白名单 API 与 `main` 进程通信；`main` 负责窗口管理与后端调用，以 `spawn`（无 shell）启动 `databackup` 子进程，stdout 按行解析 JSONL 事件，口令仅经 stdin 传递。安全三件套 `contextIsolation/sandbox/nodeIntegration=false` 不变。
- **后端（Go）**：`databackup` CLI 单二进制，读写备份仓库（`.dbk` 文件目录）、配置文件与用户文件系统。**系统不使用数据库**（ADR-005）：备份列表与详情由扫描 `.dbk` 文件头尾得出，仓库即文件系统目录。

两构件间的唯一接口是 `../architecture/cli-contract.md` 冻结的 CLI/JSONL 协议，两端各有契约测试比对协议样例。

2.1.2 子系统一结构（后端核心）

2.1.2.1 构件图



（源文件 `diagrams/component-backend.puml`。）

2.1.2.2 构件图描述

后端按数据流水线分层，依赖方向自上而下（`docs/architecture/overview.md` §2）：

- **入口层**：`cli`（子命令/参数/输出/错误映射）→ `jsonl`（协议类型与 `Emitter`）、`version`。
- **编排层**：`engine` 编排 `walk → filter → pack → [huffman] → container(→[crypto])` 的备份流水线及其逆过程的还原流水线；进度节流在 engine 完成（每阶段 ≤10 次/秒，阶段结束必发一次）。
- **原语层**：`fsmeta`（文件信息模型与 Linux 元数据读写）、`walk`（先序遍历+硬链接识别）、`filter`（Include/Exclude/SkipDir 判定）、`pack`（MyPack 归档）、`huffman`（分块规范

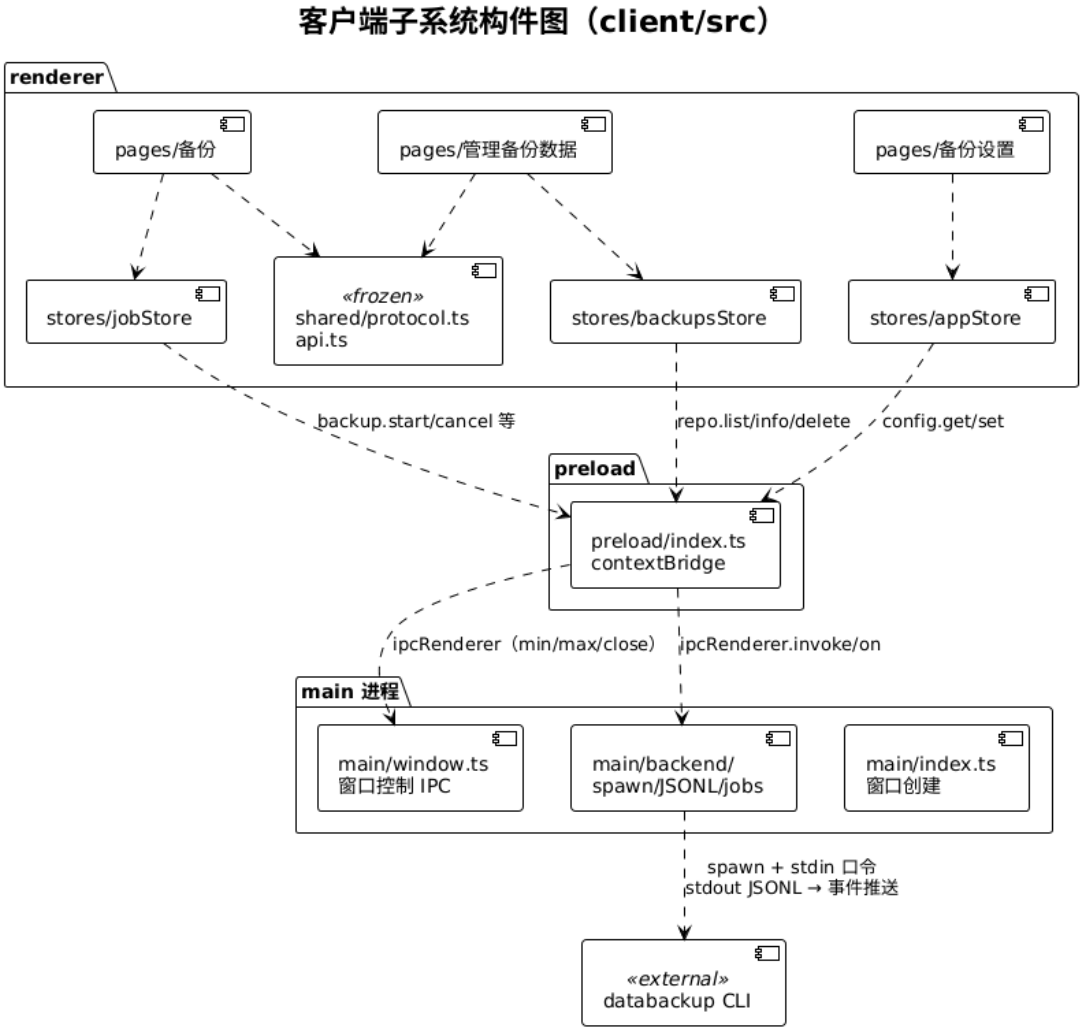
Huffman）、`crypto/sha256|hmac|pbkdf2|aes`（手写密码学原语）、`container`（`.dbk` 容器：明文头+负载+尾部统计+校验值）。

- **支撑层**：`repo`（仓库扫描/查看/删除/名称规则）、`config`（`config.json` 读写）、`testutil`（测试夹具）。

原语包不得依赖 `engine`/`cli`，可独立测试与并行开发；面向用户的错误一律为包级哨兵错误，经 `cli.Classify` 映射为契约 §4 的错误码与退出码。

2.1.3 子系统二结构（客户端）

2.1.3.1 构件图



（源文件 `diagrams/component-client.puml`。）

2.1.3.2 构件图描述

客户端三进程结构（`docs/client/interaction.md` §2）：

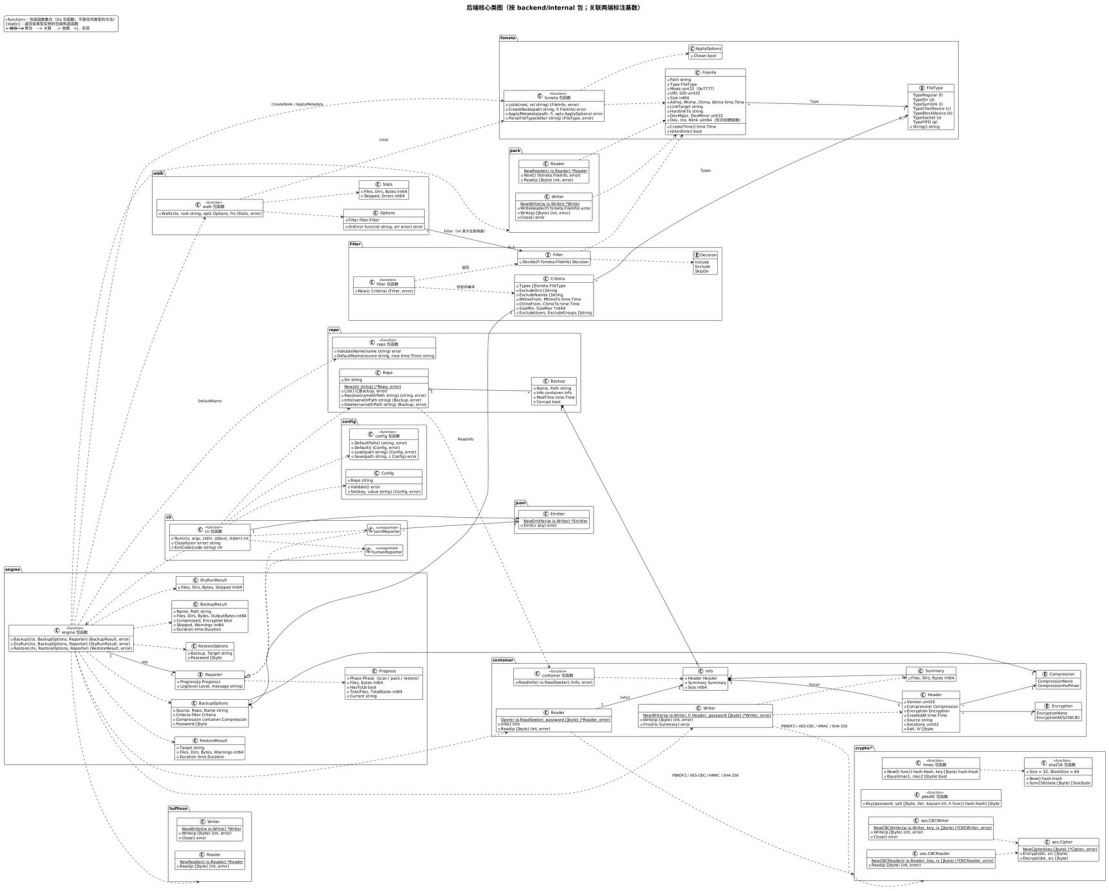
- **main**：`index.ts` 创建无边框窗口；`window.ts` 提供最小化/最大化/关闭 IPC 与最大化状态推送；`backend/` 目录维护后端调用——

`runner.ts` spawn 子进程并逐行解析 JSONL（口令写 stdin 后立即关闭、取消先 SIGTERM 3 秒后 SIGKILL）、`jobs.ts` 任务注册表（备份/还原互斥 `E_BUSY`）、`args.ts`/`resolve.ts`/`jsonl.ts`/`mock.ts` 等，转为 IPC 事件推给 renderer；`ipc/` 内 zod 校验的处理器与 `dialogs.ts` 目录选择/关闭确认。

- **preload**：`contextBridge` 暴露 `window.databackup`（应用/窗口/对话框/备份/还原/任务/仓库/配置八个命名空间），方法签名与 `src/shared/api.ts` 冻结类型一致。
- **renderer**：zustand 三个 store——`jobStore`（备份/还原任务状态机，见 §4.2）、`backupsStore`（列表缓存）、`appStore`（页面导航、后端状态与版本、减少动效、侧栏折叠）；三个页面——备份、管理备份数据、设置；`src/shared/` 协议与 API 类型为冻结件，与后端 `jsonl` 类型经契约测试保持一致。

3. 静态建模

3.1 系统对象模型



（源文件 `diagrams/class-backend.puml`。按包组织；关联两端标注基数（`1`、`0..1`、`*`、`0..*`），`*--` 为组合、`o--` 为聚合、`-->` 为关联、`..>` 为依赖、`<|..` 为接口实现。Go 的包级函数不属于任何类型，图中以 `<<function>>` 伪

类汇总；返回某类型实例的包级构造函数（如 pack.NewWriter、container.Open）以 {static} 标在该类型内。主要关系：Repo "1" o-- "*" Backup（仓库聚合多个备份）；Backup "1" *-- "1" Info、Info "1" *-- "1" Header、Info "1" *-- "1" Summary（容器信息按值组合）；BackupOptions "1" *-- "1" Criteria；walk.Options "*" --> "0..1" Filter（为 nil 时全部保留）；FileInfo "*" --> "1" FileType；cli 中两个未导出类型 jsonlReporter、humanReporter 实现 engine.Reporter。）

3.2 类（对象）描述

以下类描述与 backend/internal/* 各包已冻结的公开 API（go doc ./internal/<包>）一一对应；签名照抄源码，ctx 均为 context.Context。"包函数"指 Go 包级函数，不是某个类型的方法。接口 engine.Reporter 由 cli 实现适配到 JSONL 事件或人类可读输出。

3.2.1 fsmeta.FileInfo 类（对象）

3.2.1.1 类描述

文件信息模型：walk → filter → pack → 还原全链路共用的数据模型，覆盖 Linux 全部文件类型与需要恢复的元数据。M1-3 冻结，之后只能新增字段。硬链接不是独立类型，而是 Type 为 TypeRegular 且 HardlinkTo 非空的条目。配套类型：FileType（uint8 枚举，TypeRegular…TypeFIFO 七个取值，String() 返回 --types 字母 f d l c b s p）、ApplyOptions（控制 ApplyMetadata 恢复哪些元数据）。

3.2.1.2 属性描述

属性	类型	说明
Path	string	相对源目录的路径，/ 分隔，不含 .、.. 段，不以 / 开头或结尾
Type	FileType	文件类型：f 普通 d 目录 l 软链接 c 字符设备 b 块设备 s 套接字 pFIFO
Mode	uint32	权限位及 setuid/setgid/sticky 位（低 12 位，即 0o7777），不含类型位
UID, GID	uint32	数字属主与属组

属性	类型	说明
Size	int64	普通文件的数据长度（字节）；其他类型以及硬链接条目为 0
Atime, Mtime, Ctime	time.Time	访问、修改、状态变更时间（纳秒精度）
Btime	time.Time	创建时间（statx 的 btime）；文件系统不支持时为零值
LinkTarget	string	软链接内容（原样保存，不解析）；其他类型为空
HardlinkTo	string	非空表示本条目是硬链接，值为归档中更早出现的同一 inode 条目的 Path；此时不携带数据
DevMajor, DevMinor	uint32	字符/块设备的主、次设备号；其他类型为 0
Dev, Ino, Nlink	uint64	源文件系统上的设备号、inode 号与链接数，**只用于遍历时识别硬链接，不写入归档**
`ApplyOptions.Chown`	bool	为 true 时恢复 UID/GID（需要 root）；非 root 时调用方置 false 并发 warn

3.2.1.3 方法描述

方法 / 包函数	说明
`(fi FileInfo) CreateTime() time.Time`	方法：返回"创建时间"过滤所用的时间，有 Btime 用 Btime，否则退化为 Ctime
`(fi FileInfo) IsHardlink() bool`	方法：`HardlinkTo` 非空时为 true
`(t FileType) String() string`	方法：返回 `--types` 字母，非法值返回 `?`

方法 / 包函数	说明
`ParseFileType(letter string) (FileType, error)`	包函数：把 `--types` 中的一个字母解析为 FileType
`Lstat(root, rel string) (FileInfo, error)`	包函数（Linux）：读取 `root/rel` 的信息，不跟随软链接；优先用 statx 读 btime；不填 `HardlinkTo`
`CreateNode(path string, fi FileInfo) error`	包函数（Linux）：创建特殊节点——FIFO（mkfifo）、字符/块设备与套接字（mknod）；普通文件、目录、软/硬链接不由它创建，也不设置元数据
`ApplyMetadata(path string, fi FileInfo, opts ApplyOptions) error`	包函数（Linux）：按需 lchown，再 chmod（软链接跳过），最后设置 atime/mtime（软链接设置链接本身）；Ctime、Btime 无法设置；目录应在其子项全部还原后调用

3.2.2 filter.Criteria / filter.Filter 类

3.2.2.1 类描述

`Criteria` 是用户给出的过滤条件，字段与 CLI 过滤参数一一对应，零值表示不过滤。`Filter` 是**接口**，由包函数 `New(Criteria)` 校验并编译条件后返回（内部实现为未导出的 `compiledFilter`，并发安全的纯函数），对每个 `FileInfo` 给出 `Decision`：`Include`（记录；目录还会继续进入）、`Exclude`（不记录；目录仍进入其子项）、`SkipDir`（只用于目录：不记录也不进入）。语义以 `cli-contract.md` §2.1"过滤语义"为准。

3.2.2.2 属性描述

属性	类型	说明
Types	[]fsmeta.FileType	只备份所列类型；空表示全部
ExcludeDirs	[]string	要排除的目录，相对源目录、`/` 分隔（绝对路径已由调用方换算）
ExcludeNames	[]string	正则（RE2）列表，文件名（不含目录部分）匹配任一则排除
MtimeFrom,	time.Time	修改时间闭区间，零值

属性	类型	说明
MtimeTo		表示该端不限
CtimeFrom, CtimeTo	time.Time	创建时间闭区间（使用 `FileInfo.CreateTime()`），零值不限
SizeMin, SizeMax	`*int64`	尺寸闭区间（字节），只作用于普通文件；nil 表示该端不限
ExcludeUsers, ExcludeGroups	[]string	排除的属主/属组，名称或十进制数字 ID

3.2.2.3 **方法描述**

方法 / 包函数	说明
`New(c Criteria) (Filter, error)`	包函数：编译正则、解析用户/组名为 ID、检查区间；非法（未知类型、非法排除路径、正则错误、区间颠倒、负尺寸、用户/组不存在）返回包装 `ErrInvalidCriteria` 的错误（cli 映射 E_USAGE）
`Decide(fi fsmeta.FileInfo) Decision`	接口方法：排除目录命中 → SkipDir；其他目录只由 Types 决定 Include/Exclude；非目录须同时通过类型、名字、时间、尺寸（仅普通文件）、用户/组条件

3.2.3 walk.Options / walk.Stats 类

3.2.3.1 **类描述**

先序遍历源目录，产出经过过滤的 `fsmeta.FileInfo` 序列。约定：不跟随软链接（lstat）；同一目录内按名字字节序排序，输出顺序稳定；源目录本身不输出；按 Dev+Ino 识别硬链接（只对被保留的普通文件），第二次及以后出现的路径填写 `HardlinkTo`。进度节流由 engine 负责，本包不做。

3.2.3.2 **属性描述**

类	属性	类型	说明
Options	Filter	filter.Filter	过滤器（接口）；nil 时全部保留
Options	OnError	`func(rel string, err`	处理单个条目的读取错误：返回

类	属性	类型	说明
		`error) error`	nil 跳过该条目继续，返回非 nil 中止遍历；为 nil 时任何错误都中止
Stats	Files	int64	保留的非目录条目数（含链接与特殊文件）
Stats	Dirs	int64	保留的目录数
Stats	Bytes	int64	保留的普通文件数据总字节（硬链接条目不计）
Stats	Skipped	int64	被过滤掉的条目数（SkipDir 的目录计 1，不计其内部）
Stats	Errors	int64	经 OnError 跳过的条目数

3.2.3.3 方法描述

包函数	说明
`Walk(ctx, root string, opts Options, fn func(fsmeta.FileInfo) error) (Stats, error)`	先序遍历 root，对每个保留的条目调用 fn；fn 返回错误时中止并返回该错误；ctx 取消时尽快返回 `ctx.Err()`

3.2.4 pack.Writer / pack.Reader 类

3.2.4.1 类描述

MyPack 归档的写/读器，用法同 `archive/tar`：每个对象是 `BlockSize`（512 字节，来源 tar/UStar）的头记录加补齐到 512 倍数的数据；长文件名使用多个头块；归档以两个全零块结尾。`Reader` 除拒绝绝对路径与 `..` 段外，还拒绝以已出现的软链接为祖先的路径。字节级格式见 `../architecture/mypack-format.md`。

3.2.4.2 属性描述

无公开字段；写/读状态内部维护（当前条目、剩余字节）。包级常量与哨兵错误：`BlockSize = 512`；`Magic = "MYPK"`（每个头块开头的魔数）；`Version = 1`（当前格式版本）；`MaxNameLen = 1 << 16`（路径、软链接目

标、硬链接目标的最大字节数）；`ErrCorrupt`（魔数/校验和不符、字段非法、意外截断，cli 映射 E_CORRUPT）；`ErrUnsafePath`（路径为绝对路径、含 `..` 或为空，E_CORRUPT）；`ErrWriteTooLong`（写入数据超过头记录声明的 Size）。

3.2.4.3 方法描述

方法 / 包函数	说明
`NewWriter(w io.Writer) *Writer`	包函数：返回写入 w 的 Writer
`(w *Writer) WriteHeader(fi fsmeta.FileInfo) error`	写一个条目的头记录（必要时多个头块）；普通文件（非硬链接）之后须恰好写入 `fi.Size` 字节；上一条目数据不足时报错；Path 不合法返回 `ErrUnsafePath`
`(w *Writer) Write(p []byte) (int, error)`	写当前条目数据；超过声明长度时只写到声明长度并返回 `ErrWriteTooLong`
`(w *Writer) Close() error`	补齐最后一个条目、写入结尾两个零块；不关闭底层 Writer；重复调用返回 nil
`NewReader(r io.Reader) *Reader`	包函数：返回从 r 读取的 Reader
`(r *Reader) Next() (fsmeta.FileInfo, error)`	跳过当前条目剩余数据、读下一条目头；结尾零块返回 `io.EOF`；校验失败 `ErrCorrupt`，路径不安全 `ErrUnsafePath`；返回的 Dev、Ino、Nlink 为 0
`(r *Reader) Read(p []byte) (int, error)`	读当前条目数据，读完返回 `io.EOF`

3.2.5 huffman.Writer / huffman.Reader 类

3.2.5.1 类描述

分块流式 Huffman 编解码：输入按 `BlockSize`（1 MiB）分块，每块统计频率、构造规范 Huffman 码表并编码，块头记录 256 项码长表；流以 4 字节全零的结束块结尾。`Reader` 对一切非法输入返回包装 `ErrCorrupt` 的错误，不 panic、不死循环。

3.2.5.2 属性描述

写/读器无公开字段。包级常量与哨兵错误：`BlockSize = 1 << 20`（每块最大原始长度）；`ErrCorrupt`（码表非法、截断等，cli 映射 E_CORRUPT）。

3.2.5.3 方法描述

方法 / 包函数	说明
`NewWriter(w io.Writer) *Writer`	包函数：返回把压缩结果写入 w 的 Writer
`(w *Writer) Write(p []byte) (int, error)`	缓冲数据，每满一块编码输出一次
`(w *Writer) Close() error`	编码剩余数据并写入流结束标记；不关闭底层 Writer
`NewReader(r io.Reader) *Reader`	包函数：返回从 r 读取压缩流并输出原始数据的 Reader
`(r *Reader) Read(p []byte) (int, error)`	逐块解压；读到结束块返回 `io.EOF`，格式错误返回 `ErrCorrupt`

3.2.6 crypto/sha256、crypto/hmac、crypto/pbkdf2、crypto/aes 类

3.2.6.1 类描述

手写密码学原语，签名仿标准库，以便在 `_test.go` 中与标准库对照（标准库只允许出现在测试中）。`sha256` 实现 FIPS 180-4；`hmac` 实现 RFC 2104；`pbkdf2` 实现 RFC 8018 §5.2，用于从口令派生 AES 与 HMAC 密钥；`aes` 实现 AES-256 分组密码（FIPS-197）以及 CBC 模式 + PKCS#7 填充的流式加解密（NIST SP 800-38A），不提供其他密钥长度与模式。前三个包只有包函数；`aes` 有 `Cipher`、`CBCWriter`、`CBCReader` 三个类型（均无公开字段）。

3.2.6.2 属性描述

包	常量 / 哨兵错误	说明
sha256	`Size = 32`	摘要长度（字节）
sha256	`BlockSize = 64`	分组长度（字节）
aes	`BlockSize = 16`	AES 分组长度（字节），也是 IV 长度
aes	`KeySize = 32`	AES-256 密钥长度（字节）
aes	`ErrKeySize`	密钥不是 32 字节
aes	`ErrIVSize`	IV 不是 16 字节
aes	`ErrPadding`	解密后 PKCS#7 填充非法或密文长度不是分组整数倍（cli 映射 E_CORRUPT）

3.2.6.3 方法描述

方法 / 包函数	说明
`sha256.New() hash.Hash`	包函数：返回计算 SHA-256 的 `hash.Hash`
`sha256.Sum256(data []byte) [Size]byte`	包函数：一次性计算 data 的摘要
`hmac.New(h func() hash.Hash, key []byte) hash.Hash`	包函数：以 h 为底层哈希、key 为密钥的 HMAC
`hmac.Equal(mac1, mac2 []byte) bool`	包函数：常量时间比较两个 MAC，长度不同直接返回 false
`pbkdf2.Key(password, salt []byte, iter, keyLen int, h func() hash.Hash) []byte`	包函数：用 HMAC(h) 从 password、salt 迭代 iter 次派生 keyLen 字节密钥；iter ⩽ 0、keyLen ⩽ 0 或 keyLen 超过 (2^32-1)·h().Size() 属于调用方编程错误，直接 panic
`aes.NewCipher(key []byte) (*Cipher, error)`	包函数：密钥扩展；key 不是 32 字节返回 `ErrKeySize`
`(c *Cipher) Encrypt(dst, src []byte)` / `Decrypt(dst, src []byte)`	加密/解密 src 的第一个分组写入 dst（可原地）；`Cipher` 可并发使用
`aes.NewCBCWriter(w io.Writer, key, iv []byte) (*CBCWriter, error)`	包函数：返回把 CBC 密文写入 w 的写入器
`(w *CBCWriter) Write(p []byte) (int, error)` / `Close() error`	Write 加密并写出所有完整分组，不足一组留待后续；Close 写出带 PKCS#7 填充的最后一组，不关闭底层 Writer
`aes.NewCBCReader(r io.Reader, key, iv []byte) (*CBCReader, error)`	包函数：返回从 r 读取密文的解密读取器
`(r *CBCReader) Read(p []byte) (int, error)`	读取明文；流结束返回 `io.EOF`，在流末校验并去除填充，非法返回 `ErrPadding`

3.2.7 container 相关类（Header / Summary / Info / Writer / Reader）

3.2.7.1 类描述

`.dbk` 容器格式：明文头部（魔数、版本、算法、创建时间、源目录、KDF 参数、盐、IV）+ 负载（MyPack 流，可选 Huffman，加密时为 AES-256-CBC 密文）+ 明文尾部（`Summary` 统计）+ 校验值（加密：PBKDF2-HMAC-

SHA256 派生 64 字节，前 32 字节为 AES 密钥、后 32 字节为 HMAC 密钥，校验值 = HMAC-SHA256(头部 ‖ 密文 ‖ 尾部)；不加密：SHA-256(头部 ‖ 负载 ‖ 尾部)）。`Open` 先完整校验再允许读取；`ReadInfo` 只读头尾用于 list/info，无需口令。

3.2.7.2 属性描述

类	属性	说明
Header	`Version uint16`	格式版本：读取时填写；写入时忽略，总写 `FormatVersion`（=1）
Header	`Compression Compression`、`Encryption Encryption`	负载算法：`CompressionNone`/`CompressionHuffman`；`EncryptionNone`/`EncryptionAES256CBC`（均为 `uint8` 枚举）
Header	`CreatedAt time.Time`、`Source string`	创建时间（秒精度保存）、源目录绝对路径
Header	`Iterations uint32`、`Salt, IV []byte`	只在加密时有意义：写入时由 `NewWriter` 生成（Iterations 为 0 时用 `DefaultIterations` = 100000），读取时填写
Summary	`Files, Dirs, Bytes int64`	写在尾部的统计，口径同 backup 结果
Info	`Header Header`、`Summary Summary`、`Size int64`	不校验、不解密即可读到的信息；Size 为整个文件字节数
（包级）	`Ext = ".dbk"`、`PartialExt = ".partial"`	备份文件扩展名；写入中的临时文件为 `<名称>.dbk.partial`
（包级）	`ErrAuth`、`ErrPasswordRequired`、`ErrCorrupt`、`ErrUnsupportedVersion`	分别映射 E_AUTH、E_PASSWORD_REQUIRED、E_CORRUPT、E_COR

类	属性	说明
		RUPT

3.2.7.3 方法描述

方法 / 包函数	说明
`NewWriter(w io.Writer, h Header, password []byte) (*Writer, error)`	包函数：写出头部并返回负载写入器；`h.Encryption` 非 None 时 password 必须非空，由本函数用 `crypto/rand` 生成盐与 IV 并派生密钥；为 None 时 password 必须为 nil
`(w *Writer) Write(p []byte) (int, error)`	写入负载（加密时边写边加密、边计算 HMAC）
`(w *Writer) Finish(s Summary) error`	结束负载，写出尾部 s 与校验值；不关闭底层 Writer
`ReadInfo(r io.ReadSeeker) (Info, error)`	包函数：只读头部与尾部（不校验、不需要口令）；格式错误返回 `ErrCorrupt` 或 `ErrUnsupportedVersion`
`Open(r io.ReadSeeker, password []byte) (*Reader, error)`	包函数：先完整读一遍 r 校验（加密：HMAC，失败 `ErrAuth`；未加密：SHA-256，失败 `ErrCorrupt`），通过后回到负载起点返回明文读取器；加密备份 password 为空返回 `ErrPasswordRequired`；未加密备份忽略 password
`(r *Reader) Info() Info`	返回容器的头部、尾部与文件大小
`(r *Reader) Read(p []byte) (int, error)`	读取负载明文（仍是打包流，可能已压缩），读完返回 `io.EOF`

3.2.8 engine 相关类（BackupOptions / RestoreOptions / Reporter）

3.2.8.1 类描述

编排层（只属于 W3 集成泳道）：包函数 `Backup`/`DryRun`/`Restore` 把选项对象与 `Reporter` 串联到流水线。`Reporter` 是接口（进度+日志），由 `cli` 中未导出的 `jsonlReporter`（JSONL 事件）与 `humanReporter`（stderr 文本）实现。面向用户的错误类别为包级哨兵错误：`ErrSourceNotFound`（E_NOT_FOUND）、`ErrSourceNotDir`、`ErrInvali`

dOptions（E_USAGE）、ErrExists（E_EXISTS）、ErrBackupNotFound（E_NOT_FOUND）、ErrTargetNotEmpty（E_TARGET_NOT_EMPTY）；下层包的哨兵错误原样包装上传，ctx 取消返回 context.Canceled（E_CANCELLED）。

3.2.8.2 属性描述

类	属性	说明
BackupOptions	Source, Repo, Name string、Criteria filter.Criteria、Compression container.Compression、Password []byte	Repo 不存在则创建；Name 空时用 repo.DefaultName；Criteria.ExcludeDirs 可为源内绝对路径，由 engine 换算；Password 非 nil 表示 AES-256-CBC，engine 不保留其引用
RestoreOptions	Backup, Target string、Password []byte	Backup 为已经 repo.Resolve 解析的路径；Target 必须不存在或为空；备份未加密却给了 Password 时发 warn 并忽略
BackupResult	Name, Path string、Files, Dirs, Bytes, OutputBytes int64、Compressed, Encrypted bool、Skipped, Warnings int64、Duration time.Duration	对应契约 backup result
DryRunResult	Files, Dirs, Bytes, Skipped int64	对应契约 dry-run result
RestoreResult	Target string、Files, Dirs, Bytes, Warnings int64、Duration time.Duration	对应契约 restore result

类	属性	说明
Progress	`Phase` `Phase、Files, Bytes` `int64、HasTotal` `bool、TotalFiles, TotalBytes` `int64、Current` `string`	Phase ∈ `PhaseScan/PhasePack/PhaseRestore`（"scan"/"pack"/"restore"）；HasTotal 为 true 时（pack、restore 阶段）总量有效
Level	`LevelInfo / LevelWarn / LevelError`	日志级别（int 枚举）

3.2.8.3 方法描述

方法 / 包函数	说明
`Backup(ctx, opts BackupOptions, rep Reporter) (BackupResult, error)`	包函数：先 scan 得总量，再 pack 一次流式写出 `.partial`，成功后经 `os.Link` 硬链接原子发布为 `.dbk`（遇同名 `E_EXISTS`，防并发覆盖），随后删除 `.partial`；失败或取消删除 `.partial`
`DryRun(ctx, opts BackupOptions, rep Reporter) (DryRunResult, error)`	包函数：只遍历与过滤，不写文件；忽略 Name、Compression、Password
`Restore(ctx, opts RestoreOptions, rep Reporter) (RestoreResult, error)`	包函数：先完整校验备份再写出文件；取消时不回滚，发 warn 说明目标不完整
`Reporter.Progress(p Progress)` / `Reporter.Log(level Level, message string)`	接口方法：进度与日志回调；engine 在同一 goroutine 中调用，Progress 已节流（每阶段 ≤10 次/秒，阶段结束必发一次）

3.2.9 repo.Repo / repo.Backup 类

3.2.9.1 类描述

仓库目录对象：扫描 `.dbk` 文件、读容器头尾得列表（不使用数据库或索引文件，ADR-005）；名称↔路径解析；删除只允许 `.dbk`。哨兵错误：`ErrNotFound`（E_NOT_FOUND）、`ErrInvalidName`（备份名不合法或删除目标不是 `.dbk`，E_USAGE）。

3.2.9.2 属性描述

属性	类型	说明
Repo.Dir	string	仓库目录绝对路径
Backup.Name、Backup.Path	string	文件名去掉 .dbk / 绝对路径
Backup.Info	container.Info	容器头尾信息（Corrupt 时为零值）
Backup.ModTime	time.Time	文件修改时间（损坏备份代替创建时间展示）
Backup.Corrupt	bool	头部或尾部无法解析

3.2.9.3 方法描述

方法 / 包函数	说明
New(dir string) (*Repo, error)	包函数：返回位于 dir 的仓库（转为绝对路径；不要求已存在）
(r *Repo) List() ([]Backup, error)	列出全部 .dbk（忽略 .partial 与其他文件），损坏的以 Corrupt 列出；按创建时间降序、名字升序；目录不存在时返回空切片
(r *Repo) Resolve(nameOrPath string) (string, error)	含 / 视为路径，否则视为仓库内名称（可带或不带 .dbk）；不检查文件是否存在
(r *Repo) Info(nameOrPath string) (Backup, error)	读一个备份的信息；不存在 ErrNotFound，损坏 container.ErrCorrupt
(r *Repo) Delete(nameOrPath string) (Backup, error)	删除一个 .dbk 并返回其信息（损坏的也可删除）
ValidateName(name string) error	包函数：1~200 字节，不含 / 与 NUL，不以 . 开头
DefaultName(source string, now time.Time) string	包函数：<源目录名>-<YYYYMMDD-HHMMSS>（本地时间），源为 / 时记作 root；源目录名以 . 开头去掉全部前导点（去空记 root），结果超 200 字节在 UTF-8 字符边界截短源目录名部分

3.2.10 config.Config 类

3.2.10.1 类描述

用户配置（当前仅 repo，`Keys = []string{"repo"}`），JSON 存于 `$XDG_CONFIG_HOME/databackup/config.json`（未设置时 `~/.config/databackup/config.json`）；`Save` 经同目录临时文件原子改名，并发写不损坏文件。哨兵错误 `ErrInvalid`（配置值非法或 KEY 未知，E_USAGE）。

3.2.10.2 属性描述

属性	类型	说明
Repo	string	备份仓库目录绝对路径（JSON key `repo`）

3.2.10.3 方法描述

方法 / 包函数	说明
`DefaultPath() (string, error)` / `Default() (Config, error)`	包函数：配置文件路径（XDG_CONFIG_HOME → HOME/.config）/ 默认配置（Repo 为 `~/DataBackup`）
`Load(path string) (Config, error)`	包函数：文件不存在返回 `Default()`，缺失字段用默认值补齐；非法 JSON 或非法值返回包装 `ErrInvalid` 的错误
`Save(path string, c Config) error`	包函数：校验后写同目录临时文件再原子改名，必要时创建父目录
`(c Config) Validate() error`	仓库路径须为绝对路径；已存在时必须是目录
`(c Config) Set(key, value string) (Config, error)`	把 KEY=VALUE 写入副本并校验，返回新配置；KEY 未知返回 `ErrInvalid`

3.3 其它静态建模方式（可选）

对象关系补充见 §2.1 构件图与 §5 的 `.dbk` 容器格式字段表；客户端类型关系由冻结的 `client/src/shared/protocol.ts`、`api.ts` 定义并经契约测试约束，不再另绘。

4. 动态建模

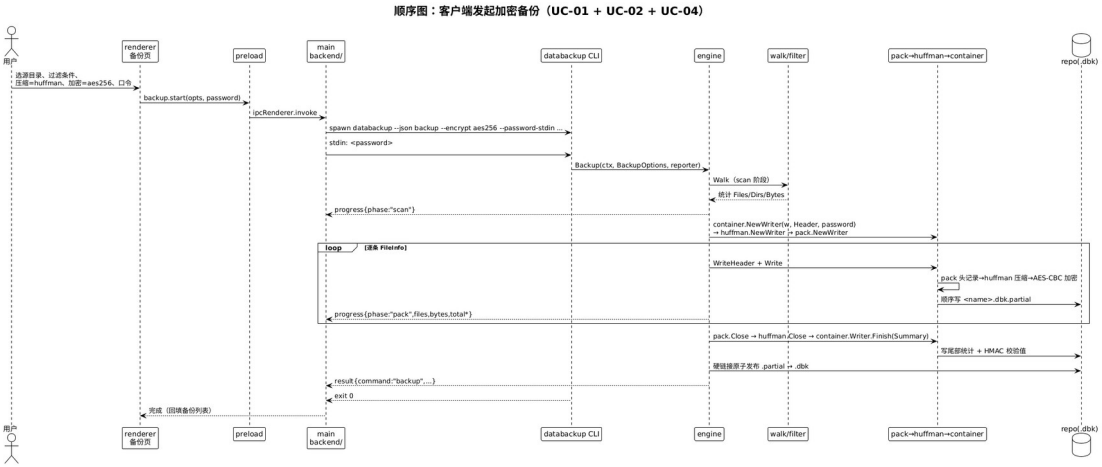
4.1 系统场景

4.1.1 场景：客户端发起加密备份（含过滤/压缩/加密）

4.1.1.1 场景描述

用户在"备份"页选择源目录、过滤条件、Huffman 压缩与 AES-256 加密并输入口令；客户端 spawn `databackup backup`，进度经 JSONL 实时回显，成功后仓库新增 `.dbk`（UC-01/02/04）。

4.1.1.2 顺序图



（源文件 `diagrams/seq-backup.puml`。）

4.1.1.3 流程说明

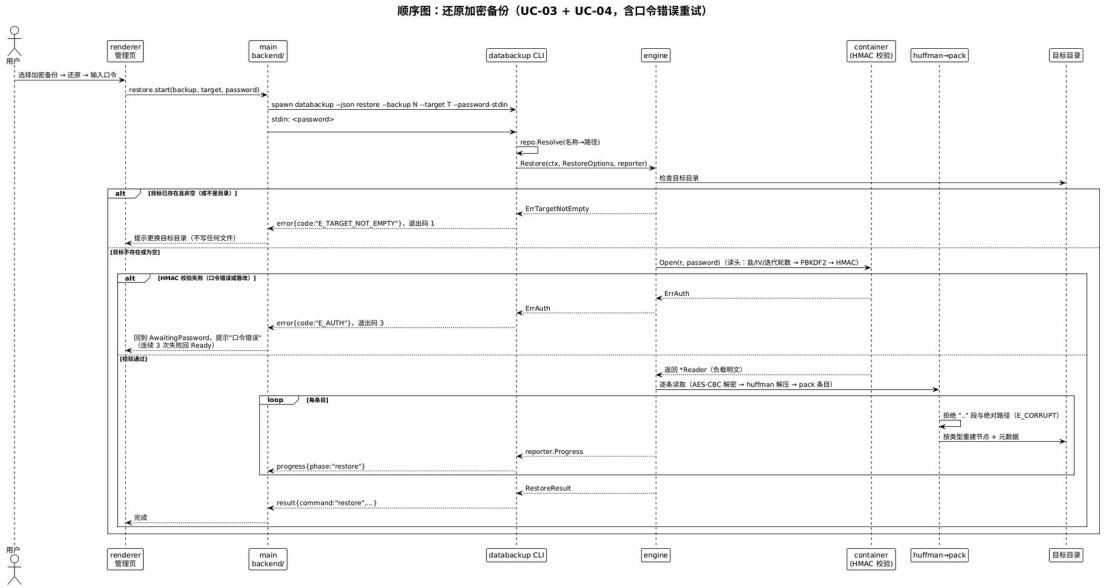
1. renderer 收集参数与口令，经 preload 调 `backup.start`；main 以 `spawn`（无 shell）启动 `databackup --json backup --encrypt aes256 --password-stdin …`，口令写入 stdin。
2. cli 解析校验参数后调 `engine.Backup`：先 scan（walk+filter 统计总量），再以 `container.NewWriter(w, Header, password)` 写出头部，逐条 `pack.WriteHeader/Write` → huffman 压缩 → AES-CBC 加密，顺序写 `<name>.dbk.partial`；progress 事件节流回传。
3. 全部写完后依次 `pack.Writer.Close`、`huffman.Writer.Close`，再 `container.Writer.Finish(Summary)` 写尾部统计与 HMAC 校验值，`.partial` 经硬链接原子发布为 `.dbk`（防并发同名覆盖）；`result` 事件输出统计；进程 exit 0，客户端回填备份列表。
4. 失败/取消：删除 `.partial`，输出对应 `error.code` 与非零退出码。

4.1.2 场景：还原加密备份（含口令校验）

4.1.2.1 场景描述

用户对加密备份发起还原；后端先校验 HMAC（口令错误 → `E_AUTH`，客户端回到口令输入态，连续 3 次失败回到就绪态），校验通过后解密解压逐条重建（UC-03/04，FR-17）。

4.1.2.2 顺序图



（源文件 `diagrams/seq-restore-encrypted.puml`。）

4.1.2.3 流程说明

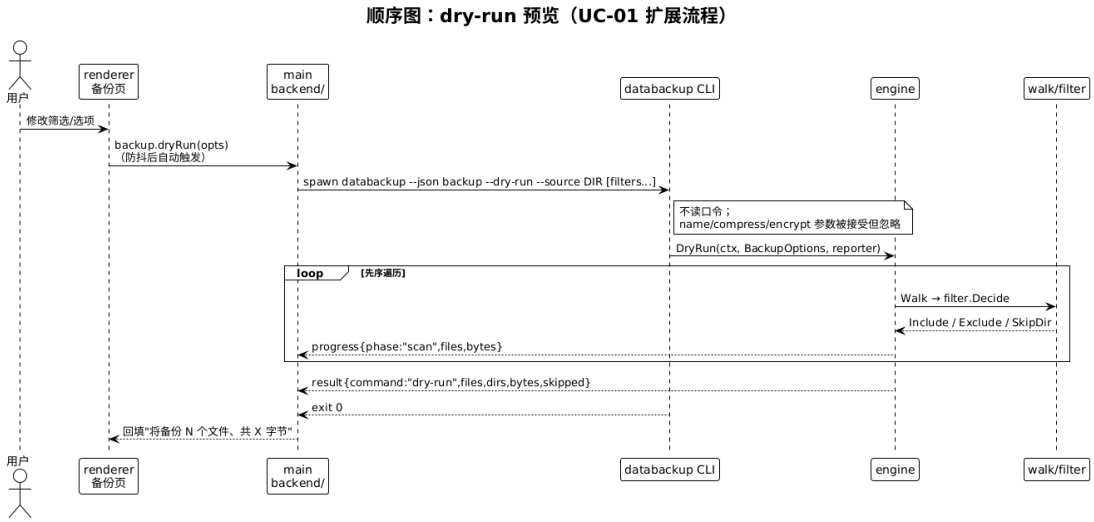
1. `cli` 先经 `repo.Resolve` 把名称解析为 `.dbk` 路径，再调 `engine.Restore`。
2. 目标目录在开始前检查：已存在且非空（或不是目录）→ `ErrTargetNotEmpty`（E_TARGET_NOT_EMPTY、退出码 1），不写任何文件。
3. `container.Open(r, password)` 读头部（盐/IV/迭代轮数）→ PBKDF2 派生密钥 → 完整读一遍做 HMAC 校验；未给口令返回 `ErrPasswordRequired`（E_PASSWORD_REQUIRED），校验失败返回 `ErrAuth`（E_AUTH），退出码均为 3，且不写任何文件。
4. 校验通过后逐条读取：AES-CBC 解密 → huffman 解压 → pack 条目；拒绝 `../`绝对路径条目（E_CORRUPT，路径穿越防护）；按类型创建节点（特殊节点用 `fsmeta.CreateNode`）并以 `fsmeta.ApplyMetadata` 恢复元数据（目录在其子项全部还原后再设置）。

4.1.3 场景：dry-run 预览

4.1.3.1 场景描述

用户修改筛选条件时，客户端防抖触发 `backup --dry-run`，只扫描统计不写文件、不读口令，用于表单下方的预览（UC-01 扩展流程）。

4.1.3.2 顺序图



（源文件 `diagrams/seq-dry-run.puml`。）

4.1.3.3 流程说明

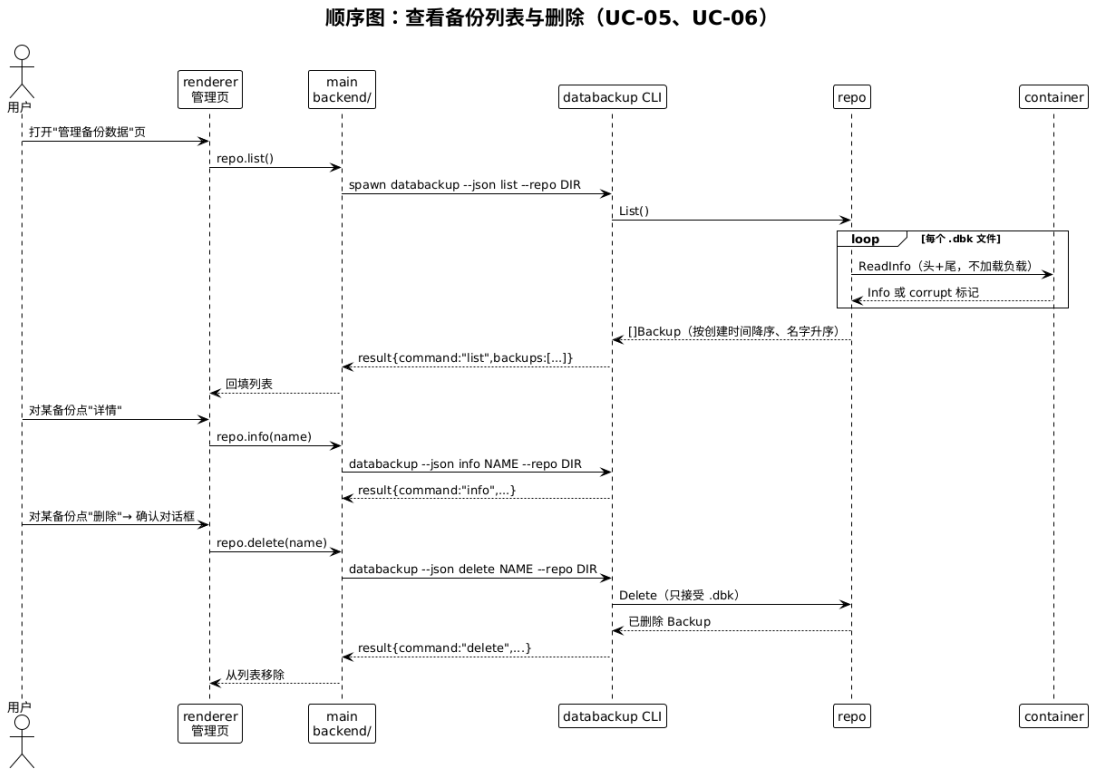
1. `cli` 接受 name/compress/encrypt 参数但忽略（契约 §2.1），不读口令。
2. `engine.DryRun` 仅执行 walk+filter：排除目录→SkipDir，目录只看 types，非目录过全部条件；progress{phase:"scan"} 回传。
3. 返回 `result{command:"dry-run",files,dirs,bytes,skipped}`，客户端展示预计规模。

4.1.4 场景：查看备份列表与删除

4.1.4.1 场景描述

"管理备份数据"页加载时调用 `list` 扫描仓库（无数据库，读容器头尾），用户可查看 `info` 详情或经确认对话框 `delete`（UC-05/06，FR-14）。

4.1.4.2 顺序图



（源文件 `diagrams/seq-list-delete.puml`。）

4.1.4.3 流程说明

1. `repo.List` 扫描 `.dbk`（忽略 `.partial` 与其他文件），逐文件 `container.ReadInfo` 读头尾；无法解析的文件以 `corrupt=true` 列出；按创建时间降序、名字升序。
2. `info` 读取单个备份的头尾元数据（不校验完整负载）。
3. `delete` 只接受 `.dbk`（显式路径必须 `.dbk` 结尾，否则 E_USAGE），客户端先二次确认。

4.2 系统状态

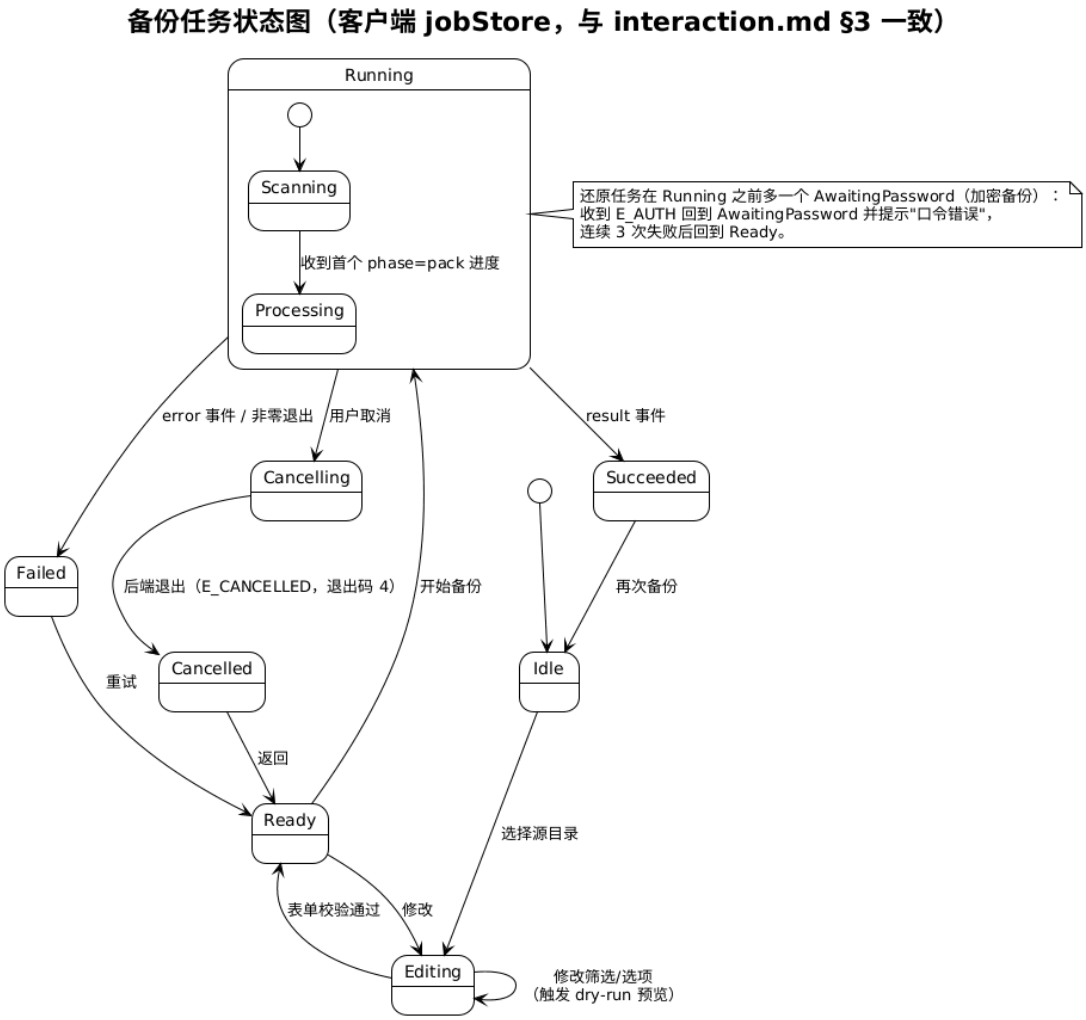
4.2.1 备份任务状态切换

4.2.1.1 状态描述

状态	含义
Idle	初始：未选源目录
Editing	编辑参数中；每次修改触发 dry-run 预览
Ready	表单校验通过，可开始
Running（Scanning/ Processing）	后端执行中；scan 进度 →Scanning，首个 pack 进度

状态	含义
	→Processing
Cancelling	用户已请求取消，等待后端退出
Succeeded / Failed / Cancelled	结果态：result 事件 / error 或非零退出 / E_CANCELLED

4.2.1.2 状态转换图



（源文件 `diagrams/state-backup-job.puml`，与 `../client/interaction.md` §3 一致。）

4.2.1.3 状态图说明

- 取消发 SIGTERM（3 秒未退出升级 SIGKILL）→ctx 取消→后端删 `.partial`、以退出码 4 结束，状态机进 Cancelled；Failed/Cancelled 均可回到 Ready 重试。
- 还原任务在 Running 之前多一个 `AwaitingPassword`（加密备份）：`E_AUTH` 回到该态并提示"口令错误"，连续 3 次失败回到 Ready。

5. 数据库设计（如有）

本系统不使用数据库（ADR-005）。持久化只有三类文件：`.dbk` 备份文件、`config.json` 配置文件、源/还原目录本身。备份仓库的"列表查询"由扫描目录下 `.dbk` 文件并读取容器头尾完成——元数据冗余在容器内，无需索引。

`.dbk` 容器格式（`container` 包，字节级字段表见 `../architecture/container-format.md`）。整数一律小端序，结构为 头部 ‖ 负载 ‖ 尾部 ‖ 校验值：

区段	字段	说明
头部（明文）	magic	格式魔数 `DBAK`
	version	格式版本（当前 `FormatVersion=1`）
	compression	负载压缩算法（none/huffman）
	encryption	负载加密算法（none/aes256-cbc）
	createdAt	备份创建时间（秒精度）
	source	备份时源目录绝对路径
	iterations, salt, IV	PBKDF2 迭代轮数（默认 100000，上限 1000000）、16 字节盐、16 字节 CBC 初始向量（仅加密，`crypto/rand` 生成）；PBKDF2 输出 64 字节，前 32 字节为 AES 密钥、后 32 字节为 HMAC 密钥
负载	MyPack 流	`pack.Writer` 产出的归档字节流，可选 Huffman 压缩、可选 AES-256-CBC 加密
尾部（明文，40 字节）	magic, payloadLen, files, dirs, bytes	尾部魔数 `DBKE`、负载长度、备份统计（同 backup 结果口径）

区段	字段	说明
校验值	HMAC-SHA256 或 SHA-256	加密：HMAC(头部 ‖ 密文 ‖ 尾部)，encrypt-then-MAC；未加密：SHA-256(头部 ‖ 负载 ‖ 尾部)

MyPack 负载内部：每条目为 512 字节头记录（路径、类型、权限、属主、时间戳、尺寸、链接目标、设备号、校验和等）+ 数据块（512 对齐），长文件名使用多头块，归档以零块结束。头块为定长小端二进制字段：魔数 `MYPK`、版本、类型码（沿用 tar 的 `0`~`6`，套接字扩展为 `7`）、CRC-32 校验和、权限/属主/设备号、长度、纳秒级 mtime/atime/ctime/btime，以及 408 字节内联名字区（放不下时溢出到扩展块）。读取时拒绝绝对路径、`..` 段和位于已出现软链接之下的路径。完整字段表见 `../architecture/mypack-format.md`。

配置存储：`$XDG_CONFIG_HOME/databackup/config.json`（未设置 XDG 时 `~/.config/...`），JSON 对象，当前仅 `repo`（绝对路径）一键；`Save` 走同目录临时文件原子改名。

软件测试报告

负责任务：M8-1 ~ M8-4。课程要求：≥2 种测试类型、≥15 个测试用例；本报告收录 24 张代表性用例表（全部用例编号内嵌于测试代码，可直接检索，见 `docs/conventions/testing.md` §3），另附覆盖率、静态检查、性能与健壮性证据。 程序版本：v0.1.0（master `a84bb18`）；除特别注明外，数据实测于 2026-10-16 ~ 2026-10-17。

1. 引言

1.1 测试目的与范围

验证 DataBackup v0.1.0 对需求说明书 FR-01 ~ FR-18 全部功能需求的实现正确性、健壮性与性能，覆盖后端 CLI、核心库与 Electron 客户端三层。

1.2 测试环境

项	值
操作系统	Ubuntu 22.04（Docker 开发镜像 / GitHub Actions runner，root）；Ubuntu 24.04（WSL2，非 root，演示环境）
Go / Node	go1.27.1 / Node.js 22.23.3（`scripts/install-deps.sh` 锁定，SHA256 校验）
测试工具	go test（单元/集成/竞态/fuzz/bench）、testscript（CLI 黑盒）、Vitest（客户端单测）、Playwright（端到端 + 界面截图回归）、staticcheck 2026.2.1、go vet、ESLint、pprof
运行方式	`make check`（单元+集成+客户端单测+静态检查）/ `make check-all`（另含 race、两端 e2e）/ CI 与本地等价

1.3 测试类型与规模

共 8 种测试类型（远超课程 ≥2 种要求）；测试代码内嵌用例编号 **400**+ 个（`TC-<模块>-<序号>`，全局可检索）。分布：单元（crypto/pack/huffman/filter/walk/fsmeta/container/repo/config）、集成（engine 流水线）、CLI 黑盒（testscript txtar + 组合矩阵）、端到端（真实二进制 SIGTERM 取消、Electron 真实后端链路）、界面回归（截图基线）、fuzz、性能（benchmark + pprof）、竞态（race）。本报告 §2/§5 按模板字段收录代表性用例 24 例。

2. 功能测试

2.1 密码学原语模块（`internal/crypto`，FR-13 底层）

字段	内容
程序版本号	v0.1.0
模块名	crypto/sha256（手写 SHA-256）
测试用例编号	TC-SHA-001
用例级别	单元测试
用例名称	空消息摘要（NIST FIPS 180-4 官方向量）
测试时间	2026-10-16（CI 于 10-17 复跑通过）
预置条件	无
测试输入	空字节串
操作步骤	调用 `sha256.Sum`，与官方期望值逐字节比对；同套件另含 abc 单块、多块向量（TC-SHA-002~009）及与标准库 `crypto/sha256` 的随机对照（TC-SHA-101~116）
预期结果	`e3b0c442...7852b855`
实际输出	一致（PASS）
测试人员	王科瑾（AI 辅助）

字段	内容
程序版本号	v0.1.0

字段	内容
模块名	crypto/aes（手写 AES-256 + CBC + PKCS#7）
测试用例编号	TC-AES-001
用例级别	单元测试
用例名称	FIPS-197 附录 C.3 分组加密向量
测试时间	2026-10-16
预置条件	无
测试输入	FIPS-197 C.3 的明文/密钥
操作步骤	分组加密后与官方密文比对；套件另含解密、逆向量、CBC 流式填充边界（TC-AES-002~024）及与标准库随机对照
预期结果	密文与 FIPS-197 一致
实际输出	一致（PASS）
测试人员	王科瑾（AI 辅助）

字段	内容
程序版本号	v0.1.0
模块名	crypto/pbkdf2（手写 PBKDF2-HMAC-SHA256）
测试用例编号	TC-KDF-001
用例级别	单元测试
用例名称	RFC 7914 §11 官方向量（passwd/salt, c=1）
测试时间	2026-10-16
预置条件	无
测试输入	口令 "passwd"、盐 "salt"、迭代 1 次
操作步骤	派生 64 字节密钥与 RFC 期望比对；另含 c=2/4096 向量（TC-KDF-002~008）
预期结果	`55ac046e...9d77ef31`
实际输出	一致（PASS）

字段	内容
测试人员	王科瑾（AI 辅助）

2.2 压缩模块（`internal/huffman`，FR-12）

字段	内容
程序版本号	v0.1.0
模块名	huffman（1MiB 分块流式规范 Huffman）
测试用例编号	TC-HUFF-001
用例级别	单元测试
用例名称	空输入编解码往返
测试时间	2026-10-16
预置条件	无
测试输入	0 字节
操作步骤	编码→解码，比对往返一致；套件另含单字节、全同字节（极限压缩）、随机、跨块大数据（TC-HUFF-002~019），非法输入返回 `ErrCorrupt`
预期结果	解码输出与输入逐字节一致
实际输出	一致（PASS）；512MB 文本实测压缩率 0.616（见 §4.2）
测试人员	王科瑾（AI 辅助）

2.3 打包模块（`internal/pack`，FR-11、FR-17）

字段	内容
程序版本号	v0.1.0
模块名	pack（MyPack v1 Reader 安全校验）
测试用例编号	TC-PACK-070
用例级别	单元测试
用例名称	拒绝绝对路径条目（路径穿越防护）
测试时间	2026-10-16
预置条件	构造含 `/etc/passwd` 条目的归档

字段	内容
测试输入	头记录路径 `/etc/passwd`
操作步骤	Reader 读取该归档
预期结果	返回损坏错误，不产出任何条目
实际输出	拒绝并报错（PASS）
测试人员	王科瑾（AI 辅助）

字段	内容
程序版本号	v0.1.0
模块名	pack
测试用例编号	TC-PACK-071
用例级别	单元测试
用例名称	拒绝含 `..` 段的相对路径
测试时间	2026-10-16
预置条件	构造含 `a/../../etc` 条目的归档
测试输入	头记录路径 `a/../../etc`
操作步骤	Reader 读取该归档
预期结果	返回损坏错误；同类防护另含硬链接目标越界（TC-PACK-074）与位于已出现软链接之下的路径
实际输出	拒绝并报错（PASS）
测试人员	王科瑾（AI 辅助）

2.4 容器模块（`internal/container`，FR-13）

字段	内容
程序版本号	v0.1.0
模块名	container（.dbk 容器格式 v1）
测试用例编号	TC-CTR-001
用例级别	单元测试
用例名称	未加密容器字节布局黄金文件
测试时间	2026-10-16
预置条件	无
测试输入	1 文件 1 目录 5 字节的明文负载

字段	内容
操作步骤	构建容器与 `testdata/plain.golden` 逐字节比对，并与 `container-format.md` §7 示例对照
预期结果	头/负载/尾统计/校验值布局与规范一致
实际输出	一致（PASS）
测试人员	王科瑾（AI 辅助）

2.5 过滤模块（`internal/filter`，FR-05 ~ FR-10）

字段	内容
程序版本号	v0.1.0
模块名	filter（六维过滤条件编译与判定）
测试用例编号	TC-FILT-001
用例级别	单元测试
用例名称	普通文件命中默认类型过滤
测试时间	2026-10-16
预置条件	无
测试输入	`a.txt`（普通文件，10 字节）
操作步骤	`filter.New` 默认条件判定；套件对类型/名字正则/时间/尺寸/排除目录/排除用户组六维各含命中、不命中与边界例（TC-FILT-002~049，共 49 例）
预期结果	Include
实际输出	Include（PASS，49/49）
测试人员	王科瑾（AI 辅助）

2.6 遍历与元数据模块（`internal/walk`、`internal/fsmeta`，FR-01、FR-03、FR-04）

字段	内容
程序版本号	v0.1.0
模块名	walk（先序遍历）
测试用例编号	TC-WALK-001

字段	内容
用例级别	单元测试
用例名称	先序与字节序排序，多次运行结果相同
测试时间	2026-10-16
预置条件	声明式目录树（普通文件、目录、软链接混合）
测试输入	混合树
操作步骤	两次遍历比对条目序列；另含软链接不跟随/不死循环、硬链接按 (Dev,Ino) 识别（TC-WALK-002~016）
预期结果	序列稳定一致
实际输出	一致（PASS）
测试人员	王科瑾（AI 辅助）

字段	内容
程序版本号	v0.1.0
模块名	fsmeta（元数据七类型）
测试用例编号	TC-FSM-001
用例级别	单元测试
用例名称	普通文件 lstat 五要素完整（类型/路径/数据/元数据/链接）
测试时间	2026-10-16
预置条件	0640 权限的普通文件
测试输入	`a.txt`（内容 hello）
操作步骤	`fsmeta.Lstat` 与期望比对；另含全部七种类型、权限位（含 setuid/setgid/sticky）、纳秒时间、btime（statx）（TC-FSM-002~035）
预期结果	类型、权限、uid/gid、a/mtime、大小全部一致
实际输出	一致（PASS）

字段	内容
测试人员	王科瑾（AI 辅助）

2.7 引擎集成（`internal/engine`，FR-01 ~ FR-04）

字段	内容
程序版本号	v0.1.0
模块名	engine（备份/还原流水线）
测试用例编号	TC-ENG-001
用例级别	集成测试
用例名称	普通文件与目录往返五要素一致（FR-01、FR-02）
测试时间	2026-10-16
预置条件	声明式目录树（7 文件 4 目录）
测试输入	基础树
操作步骤	backup → restore → `testutil.AssertTreeEqual` 按五要素比较；套件另含软/硬链接、FIFO、设备（root）、元数据恢复、压缩/加密接入（TC-ENG-002~062）
预期结果	文件数/目录数一致，五要素逐项一致
实际输出	7 文件 4 目录，全要素一致（PASS）
测试人员	王科瑾（AI 辅助）

2.8 仓库管理与配置（`internal/repo`、`internal/config`，FR-14、FR-15）

字段	内容
程序版本号	v0.1.0
模块名	repo（仓库扫描/列出）
测试用例编号	TC-REPO-001
用例级别	单元测试
用例名称	List 列出真实备份
测试时间	2026-10-16

字段	内容
预置条件	仓库目录含 1 个合法 .dbk
测试输入	仓库目录
操作步骤	`repo.New` + `List`；另含损坏文件标记 Corrupt 且不影响其他项（TC-REPO-002~013）
预期结果	列出该备份的名称/时间/源目录
实际输出	一致（PASS）
测试人员	王科瑾（AI 辅助）

字段	内容
程序版本号	v0.1.0
模块名	config（XDG 配置读写）
测试用例编号	TC-CFG-001
用例级别	单元测试
用例名称	优先使用 XDG_CONFIG_HOME
测试时间	2026-10-16
预置条件	设置 HOME 与 XDG_CONFIG_HOME
测试输入	环境变量
操作步骤	`DefaultPath` 解析；另含默认值补齐、非法路径拒绝、并发写安全（TC-CFG-002~016）
预期结果	路径为 `$XDG_CONFIG_HOME/databackup/config.json`
实际输出	一致（PASS）
测试人员	王科瑾（AI 辅助）

2.9 CLI 黑盒（testscript，FR-01 ~ FR-15）

字段	内容
程序版本号	v0.1.0
模块名	cmd/databackup（CLI）
测试用例编号	TC-CLI-300

字段	内容
用例级别	系统测试
用例名称	备份→还原全类型往返（txtar）
测试时间	2026-10-16
预置条件	txtar 声明源树与期望树
测试输入	含全部文件类型的源目录
操作步骤	真实二进制 `backup` → `restore` → 自定义命令 `treeeq` 五要素比较
预期结果	退出码 0，输出树与期望一致
实际输出	PASS（TC-CLI-301 压缩+加密往返、TC-CLI-302 错误码矩阵亦全过）
测试人员	王科瑾（AI 辅助）

字段	内容
程序版本号	v0.1.0
模块名	cmd/databackup（组合矩阵）
测试用例编号	TC-CLI-400
用例级别	系统测试
用例名称	过滤 × 压缩 × 加密 32 组合矩阵
测试时间	2026-10-16
预置条件	真实 CLI 二进制
测试输入	4 种过滤集 × 无/huffman 压缩 × 无/aes256 加密
操作步骤	每组合经真实 CLI 备份→还原，与裁剪后的期望树五要素比较，逐行校验 JSONL 输出与进度单调性
预期结果	32/32 组合往返一致
实际输出	32/32 PASS
测试人员	王科瑾（AI 辅助）

字段	内容
程序版本号	v0.1.0

字段	内容
模块名	cmd/databackup（参数校验）
测试用例编号	TC-CLI-201
用例级别	系统测试
用例名称	backup 缺少 --source 报用法错误（退出码 2）
测试时间	2026-10-16
预置条件	无
测试输入	`databackup backup`（缺必选参数）
操作步骤	运行并检查 stderr 与退出码；同类覆盖未知 flag、非法类型、口令缺加密、非法时间区间、非法名称等（TC-CLI-110~135、201~215）
预期结果	`E_USAGE`，退出码 2
实际输出	一致（PASS，参数错误矩阵全过）
测试人员	王科瑾（AI 辅助）

2.10 客户端（FR-16、FR-18）

字段	内容
程序版本号	v0.1.0
模块名	renderer/stores/job（备份任务状态机）
测试用例编号	TC-UI-001
用例级别	单元测试（Vitest）
用例名称	初始为 idle，源目录为空
测试时间	2026-10-16
预置条件	无
测试输入	初始 store
操作步骤	断言状态与表单；套件覆盖 interaction.md §3 状态机全部转移（含口令 3 次失败回退，TC-UI-001~024）
预期结果	status === 'idle'，source === ''

字段	内容
实际输出	一致（PASS，24/24）
测试人员	王科瑾（AI 辅助）

字段	内容
程序版本号	v0.1.0
模块名	client e2e（mock 后端）
测试用例编号	TC-E2E-02
用例级别	系统测试（Playwright）
用例名称	过滤+压缩+加密备份完成（mock 全流程 + 截图基线）
测试时间	2026-10-16
预置条件	DATABACKUP_MOCK=1
测试输入	源目录、排除目录正则、压缩与加密开关
操作步骤	界面编排→开始备份→运行中/完成两张截图与基线比对（动画禁用、动态元素遮罩）
预期结果	完成卡出现，截图一致
实际输出	PASS（TC-E2E-01~06 全过，10-17 CI 复跑通过）
测试人员	王科瑾（AI 辅助）

字段	内容
程序版本号	v0.1.0
模块名	client e2e（真实后端链路，C3）
测试用例编号	TC-E2E-07
用例级别	系统测试（Playwright + 真实二进制）
用例名称	过滤+压缩+加密备份 → 列表 → 口令还原，目录树原样
测试时间	2026-10-17
预置条件	独立临时 HOME/XDG 隔离环境，

字段	内容
	确定性伪随机源数据
测试输入	源树（含应排除的 node_modules）、口令
操作步骤	界面完成备份→管理页出现条目→还原（口令经 stdin）→Node fs 断言还原树与源树一致（排除目录除外）
预期结果	备份文件落盘、还原树一致、完成卡截图一致（动态统计遮罩+格式断言）
实际输出	PASS（PR #31，本地两遍 + CI 全绿）
测试人员	王科瑾（AI 辅助）

字段	内容
程序版本号	v0.1.0
模块名	client e2e（加密还原口令路径）
测试用例编号	TC-E2E-09
用例级别	系统测试（Playwright + 真实二进制）
用例名称	还原加密备份：错误口令 E_AUTH → 重试成功
测试时间	2026-10-17
预置条件	用真实 CLI 预置一个加密备份
测试输入	先错误口令，后正确口令
操作步骤	界面发起还原→口令对话框输错→断言提示"还可尝试 2 次"→输入正确口令→还原完成且树一致
预期结果	错误口令回对话框（E_AUTH，退出码 3），重试成功
实际输出	PASS
测试人员	王科瑾（AI 辅助）

3. 代码测试

3.1 静态检查（M8-4，2026-10-17 实测）

工具	范围	结果
gofmt	后端全部	零差异（`make check` 强制）
go vet	后端全部	零告警
staticcheck 2026.2.1	后端全部	零告警
ESLint + prettier	客户端全部	零告警（`make check` 强制）
禁用 import 检查	后端生产代码	通过（生产代码零第三方依赖；`archive/*`、`compress/*`、`crypto/aes` 等仅允许出现在 `_test.go` 对照组）

3.2 覆盖率（M8-2，2026-10-17 实测，`go test -coverprofile`）

包	语句覆盖率		包	语句覆盖率
crypto/sha256	100.0%		engine	87.2%
crypto/hmac	100.0%		walk	93.2%
jsonl	100.0%		repo	87.3%
crypto/aes	99.2%		huffman	94.4%
filter	99.0%		pack	95.3%
crypto/pbkdf2	97.1%		container	92.2%
fsmeta	76.2%		testutil	85.5%
config	76.2%		cmd/databackup	75.0%
cli	23.6%		version	—（常量）

核心包（engine/container/pack/huffman/crypto/walk/filter/repo）全部 ≥ 80%，满足验收标准。两点说明：`cli` 覆盖率偏低是因为其逻辑几乎全

部经 testscript 黑盒（子进程）执行，不计入包内统计，实际行为由 TC-CLI-110~135、201~215、300~313、400 全覆盖；`fsmeta/config` 的缺口主要是极少数 root 专属与平台分支。

3.3 竞态与模糊测试（M8-2）

- **race**：`go test -race ./...` 全部通过（2026-10-17，含 engine 39s、cmd 36s 长时用例）。
- **fuzz**：`pack.FuzzReader` 30s 共 **4,881,324** 次执行、`huffman.FuzzDecode` 30s 共 40,645 次执行，均无 panic、无死循环。

4. 性能测试（M8-3，2026-10-17 实测）

4.1 微基准（`go test -bench`，benchtime=2s，开发容器 Ubuntu 22.04 / go1.27.1）

基准	迭代	ns/op	吞吐
container/BenchmarkWrite/plain	576	4,102,924	255.6 MB/s
container/BenchmarkWrite/aes256	87	27,767,527	37.8 MB/s
container/BenchmarkOpenRead/plain	570	4,269,659	245.6 MB/s
container/BenchmarkOpenRead/aes256	76	30,131,455	34.8 MB/s
aes/BenchmarkEncryptBlock（16B 块）	7,194,228	338.6	47.3 MB/s
aes/BenchmarkDecryptBlock（16B 块）	6,339,040	379.5	42.2 MB/s
aes/BenchmarkCBCWriter（流	100	23,492,016	44.6 MB/s

基准	迭代	ns/op	吞吐
式）			
aes/BenchmarkCBCReader（流式）	98	26,864,737	39.0 MB/s
hmac/BenchmarkHMACSHA256	471,937	4,884	209.7 MB/s
pbkdf2/BenchmarkPBKDF2（口令派生一次）	10	213,692,832	≈214 ms
sha256/BenchmarkSum256/1KiB	609,027	4,135	247.6 MB/s
sha256/BenchmarkSum256/1MiB	588	4,051,344	258.8 MB/s

4.2 场景吞吐与压缩率（WSL2 Ubuntu 24.04，真实二进制，NVMe）

场景	数据量	耗时	吞吐	输出	压缩率
512MB 单文件（重复文本），备份无压缩	512 MiB	3.22 s	≈167 MB/s	536,872,583 B	1.000
512MB 单文件（重复文本），备份 huffman	512 MiB	5.51 s	≈97 MB/s	330,802,542 B	**0.616**
512MB 单文件（随机数	512 MiB	7.76 s	≈69 MB/s	537,007,158 B	1.000（不可压缩）

场景	数据量	耗时	吞吐	输出	压缩率
据），备份 huffman					
10,000 个小文件（10 目录 × 1000），备份 huffman	260 KB	0.16 s	—	1,961,065 B	7.5（包头开销）
还原：512MB 文本（huffman）	512 MiB	5.67 s	≈95 MB/s	—	—
还原：10,000 个小文件	260 KB	0.36 s	—	—	—

要点：压缩备份吞吐 ≈97 MB/s，瓶颈为单线程 Huffman 编码（无压缩可达 ≈167 MB/s）；加密容器写入 ≈38~45 MB/s（AES-CBC 链式）；小文件场景每文件摊薄开销 ≈16 µs（含 lstat、包头、元数据）。

4.3 性能剖析（pprof）

对加密容器写入路径（`container` aes256，benchtime=3s）采集 CPU profile，火焰图见 `diagrams/perf-flame-aes-cbc.png`：热点集中于 AES 轮函数（S 盒查表）与 CBC 链式异或，符合手写实现的预期分布。注：WSL2 内核不提供 `perf`（`perf_event` 不可用），以 Go 原生 pprof 完成等效分析（课程推荐项的等效替代）。

5. 健壮性测试

字段	内容
程序版本号	v0.1.0
模块名	pack（篡改检测）
测试用例编号	TC-PACK-042
用例级别	单元测试
用例名称	校验和不符（名字被篡改）

字段	内容
测试时间	2026-10-16
预置条件	合法归档字节
测试输入	篡改头记录名字字段（不更新校验和）
操作步骤	Reader 读取；矩阵另含魔数/版本/其它字段篡改（TC-PACK-040~064）
预期结果	检出并报损坏错误
实际输出	检出（PASS，篡改矩阵全过）
测试人员	王科瑾（AI 辅助）

字段	内容
程序版本号	v0.1.0
模块名	container（encrypt-then-MAC）
测试用例编号	TC-CTR-020
用例级别	单元测试
用例名称	错误口令返回 ErrAuth
测试时间	2026-10-16
预置条件	加密容器与正确口令
测试输入	错误口令
操作步骤	`container.Open` 先完整校验再返回明文流；矩阵另含篡改任一字节、截断（TC-CTR-030~071）
预期结果	`ErrAuth`（CLI 层映射 `E_AUTH`，退出码 3）
实际输出	一致（PASS）
测试人员	王科瑾（AI 辅助）

字段	内容
程序版本号	v0.1.0

字段	内容
模块名	CLI 全链路错误码（M6-3）
测试用例编号	TC-CLI-310
用例级别	系统测试
用例名称	list/info/delete/config 全链路与错误码矩阵
测试时间	2026-10-16
预置条件	含正常与损坏备份的仓库
测试输入	各命令的正常/异常参数组合
操作步骤	testscript 校验 JSONL 输出与退出码（TC-CLI-310~313）
预期结果	错误码与契约一致；损坏项不影响其他备份列出
实际输出	全过（PASS）
测试人员	王科瑾（AI 辅助）

字段	内容
程序版本号	v0.1.0
模块名	engine / client e2e（取消）
测试用例编号	TC-E2E-08
用例级别	系统测试（Playwright + 真实二进制）
用例名称	备份进行中取消：UI 已取消态，仓库无 .partial 残留
测试时间	2026-10-17
预置条件	足够慢的源树（8000 小文件 + 32MB）
测试输入	备份启动后点取消
操作步骤	等首个进度事件→取消→断言 UI 已取消态、仓库无成品/半成品；SIGTERM 路径另由后端 e2e TC-E2E-101/102 覆盖（退出码 4、无残留、还原取消先发"目标不完整"警告）

字段	内容
预期结果	取消成功，无 `.partial` 残留
实际输出	PASS
测试人员	王科瑾（AI 辅助）

另：fuzz（§3.3）与 race 检测作为持续健壮性证据；`.partial` 原子改名、发布防覆盖（`E_EXISTS`）、非空目标目录拒绝（`E_TARGET_NOT_EMPTY`）均含于上述矩阵。

6. 测试结果分析

6.1 汇总

测试类型	用例规模	结果
单元测试（后端 9 包 + 客户端）	300+	全部通过
集成测试（engine）	36	全部通过
CLI 黑盒（testscript + 组合矩阵）	60+	全部通过
端到端（后端 e2e + Playwright 9 例）	13	全部通过
界面回归（截图基线）	8 张基线	全部通过
fuzz / race / 静态检查 / 覆盖率	见 §3	全部通过，核心包 ≥80%
性能	见 §4	达到设计预期

6.2 FR ↔ TC 追踪表

FR	需求	代表用例
FR-01	数据备份	TC-ENG-001、TC-CLI-300
FR-02	数据还原	TC-ENG-001、TC-CLI-300、TC-E2E-07
FR-03	文件类型支持	TC-FSM-001~035、TC-CLI-300（全类型）、TC-ENG-020 系列（链接/FIFO/设备）
FR-04	元数据支持	TC-ENG-030 系列

FR	需求	代表用例
		（权限/属主/纳秒时间恢复）、TC-FSM-001
FR-05	过滤：路径	TC-FILT 系列、TC-CLI-400、TC-E2E-07（排除目录）
FR-06	过滤：类型	TC-FILT-001~003
FR-07	过滤：名字	TC-FILT（正则维度）、TC-CLI-400
FR-08	过滤：时间	TC-FILT（时间维度）、TC-CLI-206/207
FR-09	过滤：尺寸	TC-FILT（尺寸维度）
FR-10	过滤：用户/组	TC-FILT（属主维度）
FR-11	打包解包	TC-PACK-001~078
FR-12	压缩解压	TC-HUFF-001~019、TC-CLI-400
FR-13	加密解密	TC-SHA/AES/KDF/CTR 全系、TC-CTR-020、TC-E2E-09
FR-14	备份管理	TC-REPO-001~013、TC-CLI-310~312
FR-15	设置	TC-CFG-001~016、TC-CLI-313
FR-16	图形客户端	TC-UI-001~024、TC-E2E-01~09
FR-17	安全与健壮性	TC-PACK-070~078、TC-CTR-020~071、TC-CLI 错误矩阵
FR-18	进度与日志	TC-E2E-02/07（进度展示）、TC-CLI-400（JSONL 逐行校

FR	需求	代表用例
		验）

6.3 结论与已知限制

FR-01 ~ FR-18 全部实现并有自动化用例覆盖，全部通过；静态检查零告警，核心包覆盖率 ⩾ 80%，race 与 fuzz 干净，性能达到设计预期。已知限制（不影响评分项）：

1. 非 root 环境无法创建设备文件与 chown，相关用例在非 root 下自动跳过并 warn（root 容器/CI 中真实执行）；
2. 完成卡"输出文件/压缩率"显示值随打包流内环境元数据（mtime/uid）微幅漂移，属预期（见 PR #31 分析），界面 e2e 以格式断言覆盖；
3. WSL2 不提供 perf，性能剖析以 pprof 完成（课程推荐项的等效替代）。

七、AI 开发工具使用总结

本项目全程采用 AI 辅助开发：需求文档、架构设计、全部功能代码、测试用例与打包发布均由编码 agent 完成，本人承担任务拆分、审查决策与验收。

7.1 使用的 AI 工具

工具	用途	使用方式
编码 agent（Devin 等，"半强模型"）	各功能模块的实现：W1/W2 泳道任务（CLI 框架、MyPack、Huffman、密码学原语、页面等）	每任务独立 worktree + 分支；仓库根 AGENTS.md 注入项目规则；任务 ID 驱动（`docs/roadmap.md`），PR + CI 门禁合入
编码 agent（ZCode / Claude 系，高强度模型）	强节点任务与冲刺：W3 集成泳道（engine 全流水线）、C3 联调、C4 打包、测试报告成文；以及深度审查（review-agent 循环：并行深审 subagent + 门禁复跑 + P0~P3 分级报告）	同上；复杂问题拆分为并行 subagent（如截图基线跨环境漂移的根因取证）
AI 平台（阿里云百炼）	课程要求的 AI 训练证书与 API 体验	本人自行完成（H2），agent 不参与平台操作

【H7 截图 1：AI 编码 agent 执行任务的过程截图（worktree + roadmap 任务分配）】

【H7 截图 2：agent 生成的复审报告（docs/plans/*-review.md）与门禁输出】

【H7 截图 3：阿里云 AI 训练证书】

7.2 使用方法与效果评价

- **方法**：仓库以 `AGENTS.md` 为全局规则入口（硬性约束：扩展功能必须自研、生产代码零第三方依赖、不提交学号凭据等），任何 agent 进入仓库即受约束；`docs/roadmap.md` 将 40 余个任务按依赖分波次、按模型

强度分级（☒强/☒中/☒普通）调度；每个任务"先写测试再写实现"，`make check` / `check-all` 与 CI 全绿才可合并。

- **效果**：两日冲刺（2026-10-16 ~ 10-17）完成 W3 集成 → C3 联调 → C4 打包 → 测试证据全部工作，功能冻结较计划提前一天；全仓内嵌 400+ 个测试用例编号，核心包覆盖率 87%~100%，静态检查零告警；AI 产出经 review-agent 循环审查，发现并修复了多处深层问题（如 e2e 截图基线随环境元数据漂移的根因）。整体上，AI 辅助显著压缩了工期，但质量依赖"规则约束 + 分级调度 + 独立审查 + 自动化门禁"四道闸门，缺一不可。

八、实验结论

功能选择：基础要求（数据备份/还原）+ 全部扩展项——文件类型（FR-03）、元数据（FR-04）、六维过滤（FR-05~10）、打包（FR-11）、压缩（FR-12）、加密（FR-13）、图形界面（FR-16），另加仓库管理、配置、安全健壮性、进度日志四项支撑功能（FR-14/15/17/18）。

技术实现：

功能	技术
后端	Go 1.27（编译型，生产代码零第三方依赖；扩展功能全部自研）
归档	自研 MyPack 格式：512 字节小端头记录、CRC-32、长路径扩展块、双零块结尾，Reader 拒绝绝对路径/`..`/链接下路径
压缩	自研 Huffman：1MiB 分块流式规范编码，块头含码长表
加密	手写 SHA-256 / HMAC / PBKDF2 / AES-256-CBC + PKCS#7；容器 encrypt-then-MAC，错误口令退出码 3
客户端	Electron + React + TypeScript：三页面、口令经 stdin 传递、JSON Lines 实时进度
打包	electron-builder 产出 AppImage/deb，内置后端二进制；WSL2+WSLg 运行说明

最终效果：FR-01~18 全部实现并通过自动化测试（400+ 用例，32 组合过滤×压缩×加密矩阵全过，端到端界面真实链路验证）；512MB 文本备份压缩率 0.616、吞吐 ≈97MB/s，10,000 个小文件备份 0.16s；race 检测与 fuzz（488 万次执行）干净，静态检查零告警。产物：`DataBackup-0.1.0.AppImage`、`databackup-client_0.1.0_amd64.deb` 与独立 CLI 二进制。

九、总结及心得体会

【草稿，请本人润色后定稿】

1. 软件工程的"过程纪律"比单点技术更决定交付质量：本项目把需求（FR 编号）、任务（roadmap 泳道）、验证（用例编号内嵌代码）三者全程可追踪，任何一处行为变更都能回溯到需求与测试，这是两日冲刺能够不失控的根本原因。
2. 自研底层（格式/压缩/加密）的价值在于"逼真"地经历了完整设计：公开测试向量（FIPS-197、RFC 等）作为外部锚点，使手写实现可信，这一方法论可迁移到任何需要可信组件的场景。
3. AI 辅助开发的正确定位是"高强度执行者 + 纪律接受者"：规则（AGENTS.md）、分级调度与独立审查构成闸门，AI 的产能才能转化为质量而不是返工。
4. 跨端工程（Go 后端 + Electron 客户端）中，冻结接口契约（CLI/JSONL 协议文档 + 双端类型一致性测试）使两线并行开发几乎零集成冲突。

十、对本实验过程及方法、手段的改进建议

【草稿，请本人润色后定稿】

1. 契约先行可以更早落自动化：若 W0 就建立"文档 ↔ Go 类型 ↔ TS 类型"三方一致性测试，可省去中期的契约补遗 PR。
2. 性能剖析宜随开发持续进行而非收尾补做：pprof 火焰图若在 Huffman/AES 实现当周采集，可更早发现单线程瓶颈并评估并行化。
3. 演示环境的图形栈（WSLg/字体/截图基线）跨机器差异大，建议项目初期就固定"基线生成环境 = CI 环境"，避免基线漂移排查成本。
4. 对单人 + AI 组队而言，人工任务（视频录制、平台证书等）是不可并行化的瓶颈，应与代码冲刺同周启动。